

AI/ML Engineer — Complete Learning Book

A Week-by-Week, Zero-to-Job-Ready Course — 16 Weeks, Every Topic Taught
in Depth

Personal Study Companion

oday

Contents

1	How to Use This Book	5
2	Week 1 — Python Fundamentals	6
2.1	1.1 Why Python for AI/ML	6
2.2	1.2 Variables, Types, and Operators	6
2.3	1.3 Control Flow	6
2.4	1.4 Data Structures — The Ones You'll Use Every Day	7
2.5	1.5 Functions	8
2.6	1.6 Object-Oriented Programming (OOP)	8
2.7	1.7 Error Handling	9
2.8	1.8 File Handling	10
2.9	1.9 Modules and Environments	10
2.10	Practice	10
3	Week 2 — Python for Data: NumPy, Pandas, Visualization, SQL, Git	11
3.1	2.1 NumPy — The Foundation of Numerical Python	11
3.2	2.2 Pandas — Tabular Data	12
3.3	2.3 Visualization — Matplotlib and Seaborn	13
3.4	2.4 SQL Essentials	14
3.5	2.5 Git and GitHub Basics	14
3.6	Practice	15
4	Week 3 — Math I: Linear Algebra + Probability	16
4.1	3.1 Why Linear Algebra Is the Language of ML	16
4.2	3.2 Vectors	16
4.3	3.3 Matrices	16
4.4	3.4 Eigenvalues and Eigenvectors	17
4.5	3.5 Probability Fundamentals	18
4.6	3.6 Key Probability Distributions	18
4.7	Practice	19
5	Week 4 — Math II: Calculus + Statistics, and Intro to Machine Learning	20

5.1	4.1 Derivatives — The Rate of Change	20
5.2	4.2 Partial Derivatives and Gradients	20
5.3	4.3 Gradient Descent — Worked Through	20
5.4	4.4 Descriptive Statistics	21
5.5	4.5 Statistical Inference	21
5.6	4.6 Introduction to Machine Learning	22
5.7	Practice	22
6	Week 5 — Regression	24
6.1	5.1 Linear Regression — Building It From First Principles	24
6.2	5.2 Gradient Descent Variants	25
6.3	5.3 Logistic Regression	25
6.4	5.4 Regularization	26
6.5	Practice	26
7	Week 6 — Classification Algorithms	28
7.1	6.1 K-Nearest Neighbors (KNN)	28
7.2	6.2 Naive Bayes	28
7.3	6.3 Decision Trees	29
7.4	6.4 Support Vector Machines (SVM)	29
7.5	6.5 Evaluation Metrics for Classification	30
7.6	Practice	31
8	Week 7 — Ensemble Methods	32
8.1	7.1 Why Combine Models At All	32
8.2	7.2 Bagging (Bootstrap Aggregating)	32
8.3	7.3 Random Forest	32
8.4	7.4 Boosting	32
8.5	7.5 XGBoost, LightGBM, CatBoost	33
8.6	7.6 Hyperparameter Tuning	34
8.7	Practice	34
9	Week 8 — Unsupervised Learning, Feature Engineering, and Project 1	36
9.1	8.1 K-Means Clustering	36
9.2	8.2 Hierarchical Clustering	36
9.3	8.3 DBSCAN (Density-Based Clustering)	37
9.4	8.4 PCA — Full Walkthrough	37
9.5	8.5 t-SNE and UMAP (Visualization-Focused Dimensionality Reduction)	37
9.6	8.6 Feature Engineering	38
9.7	8.7 Project 1 — End-to-End ML Project (Guided Structure)	39
9.8	Practice	39
10	Week 9 — Neural Network Fundamentals	40
10.1	9.1 From a Single Neuron to a Network	40
10.2	9.2 Why Non-Linear Activations Are Non-Negotiable	40
10.3	9.3 Activation Functions in Depth	40
10.4	9.4 Forward Propagation, Concretely	41
10.5	9.5 Backpropagation — The Full Mechanics	41

10.69.6 Optimizers	42
10.79.7 Weight Initialization and Batch Normalization	42
10.89.8 PyTorch — Practical Implementation	43
10.9Practice	44
11Week 10 — Convolutional Neural Networks (CNNs)	45
11.110.1 Why Fully-Connected Layers Fail on Images	45
11.210.2 The Convolution Operation	45
11.310.3 Pooling	46
11.410.4 The Feature Hierarchy — Why Depth Matters	46
11.510.5 Classic Architectures	46
11.610.6 Transfer Learning	47
11.7Practice	48
12Week 11 — Sequence Models: RNN, LSTM, GRU	49
12.111.1 Why Sequences Need a Different Architecture	49
12.211.2 Recurrent Neural Networks (RNNs)	49
12.311.3 Backpropagation Through Time (BPTT) and the Vanishing Gradient Problem	49
12.411.4 LSTM (Long Short-Term Memory)	50
12.511.5 GRU (Gated Recurrent Unit)	50
12.611.6 Word Embeddings	51
12.711.7 A First Look at Attention (Bridging Into Next Week)	51
12.8Practice	52
13Week 12 — Transformers, NLP, and Project 2	53
13.112.1 Self-Attention — The Core Mechanism	53
13.212.2 Multi-Head Attention	54
13.312.3 Positional Encoding	54
13.412.4 The Full Transformer Block	54
13.512.5 Encoder-Only vs Decoder-Only Architectures	54
13.612.6 Fine-Tuning with Hugging Face	55
13.712.7 Project 2 — NLP Project with a Pretrained Transformer	56
13.8Practice	56
14Week 13 — DSA Foundations + Deployment Basics	57
14.113.1 Arrays and Strings	57
14.213.2 Hashing	57
14.313.3 Sorting and Binary Search	58
14.413.4 Trees and Graphs — DFS and BFS	58
14.513.5 Dynamic Programming — How to Actually Recognize and Approach It	60
14.613.6 Model Serving Basics	60
14.7Practice	61
15Week 14 — DSA Continued + Full ML Theory Review	62
15.114.1 More Tree and Graph Patterns	62
15.214.2 Backtracking	62

15.314.3 Complexity Analysis — A Practical Checklist	63
15.414.4 ML Theory — Full Review, Interview-Ready	63
15.5Practice	64
16Week 15 — ML System Design + Portfolio	66
16.115.1 Why System Design Is a Distinct Skill From Modeling	66
16.215.2 A Structured Framework You Can Reuse for Any Design Problem .	66
16.315.3 Worked Case Study — Recommendation System	67
16.415.4 Worked Case Study — Fraud Detection	68
16.515.5 What Interviewers Are Actually Listening For	68
16.615.6 Portfolio and Resume	68
16.7Practice	68
17Week 16 — Mock Interviews, Final Review, and Applications	70
17.116.1 Why This Week Is Different	70
17.216.2 Structuring a Mock Interview	70
17.316.3 Behavioral Interview Prep — The STAR Method	70
17.416.4 A Consolidated Rapid-Review Pass	70
17.516.5 Applying — Don't Wait for "Fully Ready"	71
17.6Practice	71
18Closing	73

1 How to Use This Book

This book follows your 4-month, 16-week roadmap exactly, week by week. Every topic from that roadmap is taught here in full — the goal is that you never need to leave this document to understand a concept. Where a course or video would normally show you something, this book explains it instead: the idea, the math worked out step by step, working code, and a concrete example.

How to study each week's chapter:

1. Read the chapter once fully, slowly — don't skip the math.
2. Re-derive any formula by hand on paper without looking.
3. Run the code examples yourself, then modify them (change a parameter, break something on purpose, see what happens).
4. At the end of the week, write a half-page summary in your own words before moving on.

Each chapter ends with a short **Practice** section — small, concrete exercises to lock in the concept before you move to the next week.

2 Week 1 — Python Fundamentals

2.1 1.1 Why Python for AI/ML

Python dominates ML because of its readable syntax and its ecosystem — NumPy, Pandas, scikit-learn, PyTorch, TensorFlow are all Python-first. You're not just learning a language; you're learning the entry point to every tool you'll use for the next four months. Every concept below is chosen because it shows up constantly in ML code, not because it's "core CS theory."

2.2 1.2 Variables, Types, and Operators

Python is **dynamically typed** — you don't declare a variable's type; it's inferred at runtime, and can change:

```
x = 5          # int
x = "hello"    # now a string — totally legal
x = 5.0        # now a float
```

Core built-in types: int, float, str, bool, list, tuple, dict, set, None.

Arithmetic operators: + - * / // % ** — note / always returns a float ($7/2 = 3.5$), // is floor division ($7//2 = 3$), % is modulo (remainder), ** is exponentiation.

Comparison/logical: == != > < >= <=, and, or, not (not &&/|| like other languages).

Truthiness: 0, 0.0, "", [], {}, None, and False are all "falsy" — they evaluate to False in an if check. Everything else is "truthy." This is used constantly: if my_list: checks whether the list is non-empty.

2.3 1.3 Control Flow

```
# if / elif / else
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"

# for loop — iterates over any "iterable"
for i in range(5):          # 0,1,2,3,4
    print(i)

for item in ["a", "b", "c"]:
    print(item)

# while loop
```

```

n = 5
while n > 0:
    print(n)
    n -= 1

# break / continue
for i in range(10):
    if i == 3:
        continue    # skip this iteration
    if i == 7:
        break        # exit the loop entirely
    print(i)

```

2.4 1.4 Data Structures — The Ones You’ll Use Every Day

Lists — ordered, mutable, allow duplicates:

```

nums = [3, 1, 4, 1, 5]
nums.append(9)    # [3,1,4,1,5,9]
nums.sort()       # sorts in place
nums[0]           # indexing – first element
nums[-1]          # last element
nums[1:3]         # slicing – elements at index 1,2

```

List comprehensions — the single most-used Python idiom in data work. It’s a compact way to build a new list by transforming/filtering another:

```

squares = [x**2 for x in range(10)]    # [0,1,4,9,...]
evens = [x for x in range(10) if x % 2 == 0]    # filter
pairs = [(x, y) for x in range(3) for y in range(3)]    # nested

```

Read it right-to-left in your head: “for x in range(10), give me x squared.”

Dictionaries — key-value pairs, unordered (in older Python) but insertion-ordered since 3.7+:

```

person = {"name": "Ravi", "age": 25}
person["city"] = "Ahmedabad"    # add a key
person.get("age")                # 25
person.get("salary", 0)         # 0 – default if key missing (avoids KeyError)
for key, value in person.items():
    print(key, value)

```

Dictionaries are how you’ll represent a row of features, a config, or JSON data constantly in ML code.

Sets — unordered, unique elements, fast membership testing:

```

s = {1, 2, 3}
s.add(4)
3 in s    # True – O(1) average lookup, much faster than checking a list

```

Tuples — like lists but immutable, used for fixed-size groupings (e.g., a coordinate, or a function returning multiple values):

```
point = (3, 4)
x, y = point      # unpacking
```

2.5 1.5 Functions

```
def greet(name, greeting="Hello"):    # "greeting" has a default value
    return f"{greeting}, {name}!"

greet("Ravi")                        # "Hello, Ravi!"
greet("Ravi", "Hi")                  # "Hi, Ravi!"
greet(name="Ravi", greeting="Hey")  # keyword arguments

# *args and **kwargs — variable numbers of arguments
def total(*args):                    # args becomes a tuple of all positional args
    return sum(args)
total(1, 2, 3)                       # 6

def show_info(**kwargs):             # kwargs becomes a dict of all keyword args
    for k, v in kwargs.items():
        print(k, v)
show_info(name="Ravi", age=25)
```

You'll see `*args`, `**kwargs` constantly in ML library wrapper functions — it lets a function accept and pass through an arbitrary set of arguments without knowing them in advance.

Lambda functions — small, anonymous, one-line functions, often used inline:

```
square = lambda x: x**2
square(5)      # 25
sorted([3,1,2], key=lambda x: -x)  # sort descending
```

2.6 1.6 Object-Oriented Programming (OOP)

ML codebases are full of classes (a PyTorch Model is a class; a scikit-learn estimator is a class). You need to be comfortable reading and writing them.

```
class Dog:
    def __init__(self, name, breed):    # constructor — runs when you create an instance
        self.name = name                # instance attribute
        self.breed = breed

    def bark(self):                     # instance method — takes self
        return f"{self.name} says woof!"
```

```

def __repr__(self):                # controls how print(instance) looks
    return f"Dog({self.name}, {self.breed})"

d = Dog("Rex", "Labrador")
print(d.bark())                    # "Rex says woof!"
print(d)                          # Dog(Rex, Labrador)

```

Inheritance — a class can extend another, reusing and overriding behavior:

```

class Puppy(Dog):
    def __init__(self, name, breed, age):
        super().__init__(name, breed)    # call the parent's constructor
        self.age = age

    def bark(self):                    # override the parent's method
        return f"{self.name} yips!"

```

This exact pattern — subclass a base class, call `super().__init__()`, override a method — is precisely how you'll define custom PyTorch models: `class MyModel(nn.Module): def __init__(self): super().__init__(); ...`

@staticmethod, @classmethod, @property — decorators you'll see in ML code:

```

class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property                        # lets you call c.area instead of c.area()
    def area(self):
        return 3.14159 * self._radius ** 2

    @staticmethod                  # doesn't need access to self or the class
    def is_valid_radius(r):
        return r > 0

```

2.7 1.7 Error Handling

```

try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
except Exception as e:            # catch-all for anything else
    print(f"Unexpected error: {e}")
finally:
    print("This always runs")      # cleanup code, runs regardless of error

```

Use specific exception types when you can predict the failure (e.g., `ValueError`, `KeyError`, `FileNotFoundError`) — catching broad `Exception` everywhere hides real bugs.

2.8 1.8 File Handling

```
with open("data.txt", "r") as f:      # "with" auto-closes the file, even on error
    content = f.read()

with open("output.txt", "w") as f:
    f.write("hello\n")

import json
with open("config.json") as f:
    config = json.load(f)             # parse JSON into a Python dict
```

2.9 1.9 Modules and Environments

- `import numpy as np` — import a whole module under an alias.
- `from math import sqrt` — import a specific name.
- **Virtual environments** isolate a project's package versions from your system Python, avoiding version conflicts between projects. Create one with `python -m venv myenv`, activate it, then `pip install` packages inside it.

2.10 Practice

1. Write a function that takes a list of numbers and returns a new list with only the even ones, using a list comprehension.
2. Write a `BankAccount` class with `deposit()`, `withdraw()` (raise an exception if funds are insufficient), and a `balance` property.
3. Read a `.txt` file line by line and count how many lines contain a given word.
4. Predict the output of this code before running it: `def f(x, lst=[]): lst.append(x); return lst` called three times as `f(1)`, `f(2)`, `f(3)` — then explain *why* the output surprises most beginners.

3 Week 2 — Python for Data: NumPy, Pandas, Visualization, SQL, Git

3.1 2.1 NumPy — The Foundation of Numerical Python

Every ML library (Pandas, scikit-learn, PyTorch) is built on top of NumPy arrays. A NumPy array (ndarray) stores data of a **single, fixed type** in a contiguous block of memory — this is what makes it fast, unlike a Python list, which stores pointers to scattered Python objects.

```
import numpy as np

a = np.array([1, 2, 3, 4])
a.shape      # (4,)
a.dtype      # dtype('int64')

m = np.array([[1, 2], [3, 4]])
m.shape      # (2, 2)

np.zeros((3, 3))      # 3x3 matrix of zeros
np.ones((2, 4))       # 2x4 matrix of ones
np.eye(3)             # 3x3 identity matrix
np.arange(0, 10, 2)    # [0,2,4,6,8]
np.linspace(0, 1, 5)   # 5 evenly spaced points from 0 to 1
np.random.rand(3, 3)   # random uniform values in [0,1)
np.random.randn(3, 3)  # random values from standard normal distribution
```

Indexing and slicing:

```
a = np.array([10, 20, 30, 40, 50])
a[1:4]      # [20, 30, 40]
a[::-1]     # reverse the array

m = np.array([[1,2,3],[4,5,6],[7,8,9]])
m[0, :]    # first row
m[:, 1]    # second column
m[1:, 1:]  # sub-matrix from row 1, col 1 onward

# Boolean masking — extremely common in ML data prep
a[a > 25]   # array([30, 40, 50])
```

Broadcasting — the most important NumPy concept: When you perform an operation between arrays of different shapes, NumPy “stretches” the smaller one to match, without physically copying data, following this rule: *dimensions are compatible if they are equal, or one of them is 1*.

```
a = np.array([1, 2, 3])
a + 5      # [6, 7, 8] — 5 is broadcast to every element

m = np.array([[1,2,3],[4,5,6]])  # shape (2,3)
```

```
v = np.array([10, 20, 30])          # shape (3,)
m + v                               # v is broadcast across both rows: [[11,22,33],[14,25,36]]
```

This is why you can write `X - X.mean(axis=0)` to mean-center every column of a dataset in one line — no loop needed.

Vectorization — why loops are slow, and how to avoid them:

```
# Slow — Python-level loop
result = []
for x in range(1000000):
    result.append(x ** 2)

# Fast — vectorized, runs in optimized C under the hood
x = np.arange(1000000)
result = x ** 2
```

Vectorized code is often 10–100x faster because it avoids the overhead of Python’s interpreter running once per element.

Matrix operations:

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

A * B          # element-wise multiplication (Hadamard product)
A @ B          # matrix multiplication (same as np.matmul(A,B) or np.dot(A,B) for 2D)
A.T            # transpose
np.linalg.inv(A)    # matrix inverse
np.linalg.det(A)    # determinant
np.linalg.eig(A)    # eigenvalues and eigenvectors
```

3.2 2.2 Pandas — Tabular Data

Pandas builds Series (1D, labeled) and DataFrame (2D, labeled rows and columns) on top of NumPy — this is what you’ll use for essentially all structured/tabular data work.

```
import pandas as pd

df = pd.read_csv("data.csv")
df.head()          # first 5 rows
df.info()          # column types, non-null counts
df.describe()      # summary stats for numeric columns
df.shape           # (rows, columns)
df.columns         # column names

df["age"]          # select one column (returns a Series)
df[["age", "salary"]] # select multiple columns (returns a DataFrame)
df.iloc[0]         # first row by position
df.loc[df["age"] > 30] # filter rows by condition
```

```
df["age"].mean()
df["age"].isna().sum()      # count missing values
df.dropna()                 # drop rows with any missing value
df.fillna(df["age"].median()) # fill missing values
```

groupby — split-apply-combine, the single most powerful Pandas pattern:

```
df.groupby("department")["salary"].mean()      # average salary per department
df.groupby("department").agg({"salary": "mean", "age": "max"})
```

Mentally, this splits the DataFrame into groups by the given column, applies a function to each group independently, then combines the results back into one output — exactly the same logic as GROUP BY in SQL.

Merging / joining:

```
pd.merge(orders, customers, on="customer_id", how="left")
```

how can be "inner" (only matching rows in both), "left" (all rows from the left table, matched where possible), "right", or "outer" (all rows from both, filling gaps with NaN).

Creating and transforming columns:

```
df["bonus"] = df["salary"] * 0.1
df["log_salary"] = np.log(df["salary"])
df["senior"] = df["age"].apply(lambda x: x > 40)      # row-wise transform (slower – pre
```

3.3 2.3 Visualization — Matplotlib and Seaborn

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.plot(x, y)                # line plot
plt.scatter(x, y)             # scatter plot
plt.hist(df["age"], bins=20)  # histogram
plt.xlabel("Age"); plt.ylabel("Count"); plt.title("Age Distribution")
plt.show()

sns.boxplot(x="department", y="salary", data=df)      # spot outliers per group
sns.heatmap(df.corr(), annot=True)                    # correlation matrix – spot multicollinearity
sns.pairplot(df)                                     # pairwise relationships between
```

Why EDA (exploratory data analysis) matters: before touching a model, you should know your data's distributions, missing values, outliers, and correlations — this drives every later decision (which features to engineer, which scaling to use, which model family is appropriate).

3.4 2.4 SQL Essentials

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
WHERE age > 25
GROUP BY department
HAVING AVG(salary) > 50000
ORDER BY avg_salary DESC;
```

- WHERE filters rows *before* grouping; HAVING filters groups *after* aggregation — this distinction is a very common interview trip-up.
- **Joins:**

```
SELECT o.order_id, c.name
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id;
```

INNER JOIN keeps only matching rows; LEFT JOIN keeps all rows from the left table, filling unmatched right-side columns with NULL.

- **Window functions** — compute a value across a set of rows related to the current row, without collapsing them like GROUP BY does:

```
SELECT name, department, salary,
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
FROM employees;
```

This ranks employees by salary *within* each department, while keeping every row (unlike GROUP BY, which would collapse rows).

3.5 2.5 Git and GitHub Basics

```
git init                # start tracking a new repo
git add file.py          # stage a file
git commit -m "message" # save a snapshot
git status              # what's changed
git log                 # history of commits
git branch feature-x    # create a new branch
git checkout feature-x  # switch to it
git merge feature-x     # merge it into current branch
git push origin main    # push to GitHub
git clone <url>         # copy a remote repo locally
```

Use branches for any new feature/experiment so main always stays in a working state. Write commit messages describing *why*, not just *what* (“fix data leakage in train/test split” beats “update code”).

3.6 Practice

1. Given a NumPy array of exam scores, compute the mean, standard deviation, and the count of scores above the mean — without using a Python for loop.
2. Load a CSV with Pandas, find the top 3 categories by average value in a numeric column, and handle any missing values sensibly.
3. Write a SQL query: for each customer, find their most recent order (hint: window function with `ROW_NUMBER()` partitioned by customer, ordered by date descending, then filter `row_number = 1`).
4. Initialize a Git repo for a small script, make three commits, create a branch, make a change, and merge it back.

4 Week 3 — Math I: Linear Algebra + Probability

4.1 3.1 Why Linear Algebra Is the Language of ML

Every dataset is a matrix. Every model's parameters are vectors/matrices. Every forward pass through a neural network is a sequence of matrix multiplications. If you understand linear algebra deeply, you'll understand *why* the code works, not just that it does.

4.2 3.2 Vectors

A vector is an ordered list of numbers, representing a point in space or a direction:

$$v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

Vector addition (element-wise): $u + v = [u_1 + v_1, u_2 + v_2, \dots]$ — geometrically, place the tail of one vector at the head of the other.

Scalar multiplication: $c \cdot v = [cv_1, cv_2, \dots]$ — stretches or shrinks the vector.

Dot product: $u \cdot v = \sum_i u_i v_i$. Geometrically, $u \cdot v = |u||v| \cos \theta$, where θ is the angle between them. This single formula tells you: - If $u \cdot v = 0$, the vectors are **orthogonal** (perpendicular). - If $u \cdot v > 0$, they point in a broadly similar direction. - If $u \cdot v < 0$, they point in broadly opposite directions.

This is exactly why cosine similarity (used in recommendation systems, NLP embeddings) is built on the dot product — it measures how aligned two vectors (e.g., two documents' word-frequency vectors) are, independent of their magnitude.

Vector norm (length): $\|v\| = \sqrt{\sum_i v_i^2}$ (this is the L2 norm — what you'd compute with the Pythagorean theorem generalized to n dimensions).

Worked example: $u = [3, 4]$. $\|u\| = \sqrt{9 + 16} = \sqrt{25} = 5$. This is literally the hypotenuse of a 3-4-5 triangle — vector norms are a direct generalization of distance.

4.3 3.3 Matrices

A matrix is a 2D array — you can think of it as a **linear transformation**: multiplying a vector by a matrix moves that vector to a new position, possibly rotating, scaling, or shearing it, but always preserving straight lines and the origin.

Matrix multiplication: for $C = AB$ where A is $(m \times n)$ and B is $(n \times p)$, the result C is $(m \times p)$, with each entry:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

i.e., entry (i, j) is the dot product of row i of A with column j of B .

Worked example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$
$$AB = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 6 \\ 3 \cdot 5 + 4 \cdot 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

This is exactly what happens in a neural network layer: a weight matrix W times an input vector x .

Important: matrix multiplication is **not commutative** — $AB \neq BA$ in general (and often BA isn't even defined if dimensions don't match).

Identity matrix I : has 1s on the diagonal, 0s elsewhere. $AI = IA = A$ — it's the matrix "do nothing" operator, the equivalent of multiplying a number by 1.

Transpose A^T : flips rows into columns. Used constantly in backpropagation to correctly orient gradients when reversing a matrix multiplication.

Inverse A^{-1} : satisfies $AA^{-1} = I$. Only square, full-rank ("non-singular") matrices have an inverse. Solving $Ax = b$ for x is (in theory) $x = A^{-1}b$ — though in practice, for large systems, you use numerically stable iterative solvers rather than literally computing the inverse.

Determinant $\det(A)$: a single number summarizing a square matrix. If $\det(A) = 0$, the matrix is **singular** — it has no inverse, and it "collapses" space into a lower dimension (e.g., a 2D transformation that squashes everything onto a line).

4.4 3.4 Eigenvalues and Eigenvectors

For a square matrix A , a non-zero vector v is an **eigenvector** with **eigenvalue** λ if:

$$Av = \lambda v$$

In words: applying the transformation A to v doesn't rotate v — it only scales it by λ . Most vectors get rotated *and* scaled by a transformation; eigenvectors are the special directions that only get scaled.

Why this matters for ML — PCA: Given a dataset X (mean-centered), its covariance matrix $\Sigma = \frac{1}{n}X^T X$ describes how features vary together. The eigenvectors of Σ point in the directions of maximum variance in the data, and the eigenvalues tell you how much variance lies along each direction. Sorting eigenvectors by eigenvalue (largest first) and keeping the top k gives you the best k -dimensional linear approximation of your data — this is PCA.

How to find eigenvalues (small matrices, by hand): solve $\det(A - \lambda I) = 0$ — this is called the **characteristic equation**.

Worked example: $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$. Since this is diagonal, the eigenvalues are simply the diagonal entries: $\lambda_1 = 2, \lambda_2 = 3$, with eigenvectors $[1, 0]$ and $[0, 1]$ respectively (the axes themselves — makes sense, since a diagonal matrix just stretches along each axis independently).

4.5 3.5 Probability Fundamentals

Sample space and events: the sample space Ω is the set of all possible outcomes (e.g., rolling a die: $\{1, 2, 3, 4, 5, 6\}$); an event is a subset of outcomes (e.g., “rolling even” = $\{2, 4, 6\}$).

Probability axioms: $0 \leq P(A) \leq 1$; $P(\Omega) = 1$; if A and B are mutually exclusive, $P(A \cup B) = P(A) + P(B)$.

Conditional probability:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

This reads: “given that B has happened, what’s the probability of A ?” It restricts the sample space to just the cases where B is true, then asks what fraction of those also satisfy A .

Independence: A and B are independent if knowing one gives you no information about the other: $P(A|B) = P(A)$, equivalently $P(A \cap B) = P(A)P(B)$.

Bayes’ Theorem — derivation and intuition: From the definition of conditional probability: $P(A \cap B) = P(A|B)P(B) = P(B|A)P(A)$. Rearranging:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

This lets you “flip” a conditional probability you know into one you want.

Worked example (the classic medical test problem): A disease affects 1% of a population. A test is 99% accurate (99% true positive rate, 99% true negative rate). If someone tests positive, what’s the probability they actually have the disease?

Let D = has disease, T = tests positive. $P(D) = 0.01$, $P(T|D) = 0.99$, $P(T|\text{not } D) = 0.01$ (false positive rate).

$$P(T) = P(T|D)P(D) + P(T|\text{not } D)P(\text{not } D) = 0.99 \times 0.01 + 0.01 \times 0.99 = 0.0198$$

$$P(D|T) = \frac{P(T|D)P(D)}{P(T)} = \frac{0.99 \times 0.01}{0.0198} = 0.5$$

Only **50%**, even with a “99% accurate” test — because the disease is so rare, the false positives from the huge healthy population roughly equal the true positives from the tiny sick population. This is the single most important intuition Bayes’ theorem gives you, and it directly explains why accuracy is a misleading metric on imbalanced datasets — the same logic reappears when evaluating a fraud/spam classifier.

4.6 3.6 Key Probability Distributions

Bernoulli distribution: models a single binary trial (success/failure) with probability p of success. $P(X = 1) = p$, $P(X = 0) = 1 - p$. This is the distribution logistic regression assumes for its output.

Binomial distribution: the number of successes in n independent Bernoulli trials, each with success probability p . Mean $= np$.

Normal (Gaussian) distribution: the bell curve, defined by mean μ and variance σ^2 :

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Central Limit Theorem: the sum (or average) of many independent random variables — regardless of their original distribution — tends toward a normal distribution as the number of variables grows. This is why the normal distribution appears everywhere in statistics: sample means, measurement errors, and aggregated features often converge to it.

4.7 Practice

1. By hand, compute the dot product and cosine similarity between $u = [1, 2, 3]$ and $v = [2, 4, 6]$. What do you notice about their direction, and why?
2. By hand, multiply $\begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix}$.
3. Verify that $[1, 0]$ and $[0, 1]$ are eigenvectors of $\begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$, and state their eigenvalues.
4. A factory's defect rate is 2%. A quality-check test correctly flags true defects 95% of the time, and incorrectly flags good items 3% of the time. If an item is flagged, what's the probability it's actually defective? (Use Bayes' theorem, following the worked example above.)

5 Week 4 — Math II: Calculus + Statistics, and Intro to Machine Learning

5.1 4.1 Derivatives — The Rate of Change

The derivative of $f(x)$ at a point tells you the slope of the tangent line there — how fast f is changing as x changes:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Rules you must know cold: - Power rule: $\frac{d}{dx}x^n = nx^{n-1}$ - Sum rule: $\frac{d}{dx}[f(x) + g(x)] = f'(x) + g'(x)$ - Product rule: $\frac{d}{dx}[f(x)g(x)] = f'(x)g(x) + f(x)g'(x)$ - **Chain rule:** $\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$ — differentiate the outer function, leave the inner one alone, then multiply by the derivative of the inner function.

Worked example (chain rule): $f(x) = (3x + 1)^2$. Let $g(x) = 3x + 1$ (inner), so $f = g^2$ (outer). $f'(x) = 2g(x) \cdot g'(x) = 2(3x + 1) \cdot 3 = 6(3x + 1) = 18x + 6$.

Why the chain rule is the single most important calculus concept in ML: a neural network is a composition of functions — layer 3 applied to (layer 2 applied to (layer 1 applied to the input)). To know how a change in an early layer's weight affects the final loss, you must chain derivatives through every layer in between. This chained computation, done systematically backward from the output, is exactly what **backpropagation** is.

5.2 4.2 Partial Derivatives and Gradients

When a function depends on multiple variables, e.g. $f(x, y)$, a **partial derivative** $\frac{\partial f}{\partial x}$ treats all other variables as constants and differentiates with respect to just x .

Worked example: $f(x, y) = x^2y + 3y$. $\frac{\partial f}{\partial x} = 2xy$ (treat y as constant). $\frac{\partial f}{\partial y} = x^2 + 3$ (treat x as constant).

The **gradient**, written ∇f , is the vector of all partial derivatives: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$. Geometrically, the gradient points in the direction of **steepest increase** of the function. This is why gradient descent moves in the *negative* gradient direction — to decrease the loss as fast as possible at each step.

5.3 4.3 Gradient Descent — Worked Through

Given a loss function $L(w)$ you want to minimize, gradient descent updates the parameter iteratively:

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w}$$

where α is the **learning rate**, controlling the step size.

Worked example: minimize $L(w) = (w - 3)^2$ (minimum obviously at $w = 3$). $\frac{dL}{dw} = 2(w - 3)$.

Start at $w = 0$, learning rate $\alpha = 0.1$: - Step 1: gradient = $2(0 - 3) = -6$. Update: $w = 0 - 0.1 \times (-6) = 0.6$. - Step 2: gradient = $2(0.6 - 3) = -4.8$. Update: $w = 0.6 - 0.1 \times (-4.8) = 1.08$. - Step 3: gradient = $2(1.08 - 3) = -3.84$. Update: $w = 1.08 + 0.384 = 1.464$.

Notice w is climbing steadily toward 3, with steps getting smaller as it gets closer (because the gradient shrinks as w approaches the minimum) — this self-slowng behavior is exactly why gradient descent converges smoothly rather than overshooting wildly, provided the learning rate isn't too large.

If α is too large: the update can overshoot the minimum and diverge (oscillate or blow up). **If α is too small:** convergence is correct but painfully slow. Tuning the learning rate is one of the most consequential hyperparameter choices you'll make in every model you train.

5.4 4.4 Descriptive Statistics

- **Mean:** $\bar{x} = \frac{1}{n} \sum x_i$ — sensitive to outliers.
- **Median:** the middle value when sorted — robust to outliers.
- **Variance:** $\sigma^2 = \frac{1}{n} \sum (x_i - \bar{x})^2$ — average squared distance from the mean.
- **Standard deviation:** $\sigma = \sqrt{\sigma^2}$ — same units as the data, easier to interpret than variance.
- **Covariance:** $\text{Cov}(X, Y) = \frac{1}{n} \sum (x_i - \bar{x})(y_i - \bar{y})$ — measures whether two variables move together (positive) or oppositely (negative).
- **Correlation:** $\rho = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$ — covariance normalized to always fall between -1 and 1, making it interpretable and scale-independent.

5.5 4.5 Statistical Inference

Hypothesis testing framework: 1. State a **null hypothesis** H_0 (e.g., “this new website design has no effect on conversion rate”) and an **alternative hypothesis** H_1 . 2. Collect data and compute a **test statistic**. 3. Compute a **p-value**: the probability of observing data at least this extreme, *assuming H_0 is true*. 4. If the p-value is below a chosen significance level (commonly 0.05), reject H_0 .

The single most common misunderstanding, stated precisely: a p-value of 0.03 does NOT mean “there’s a 3% chance the null hypothesis is true.” It means: “if the null hypothesis *were* true, there’s a 3% chance of seeing data this extreme by random chance alone.” This is a statement about the data given the hypothesis, not about the hypothesis given the data — mixing these up is the most-repeated statistics error in practice.

Confidence interval: e.g., a 95% confidence interval for a mean means: if you repeated the sampling process many times and built an interval the same way each time, about 95% of those intervals would contain the true population mean. It is a

statement about the *reliability of the procedure*, not a 95% probability that the true value lies in this *one* specific interval you computed.

Type I vs Type II error: - **Type I (false positive):** rejecting H_0 when it's actually true — concluding there's an effect when there isn't one. - **Type II (false negative):** failing to reject H_0 when it's actually false — missing a real effect. There's a direct tradeoff: making your significance threshold stricter (lowering the chance of Type I errors) increases the chance of Type II errors, all else equal.

5.6 4.6 Introduction to Machine Learning

What “learning” means here: rather than hand-coding rules, you give an algorithm data (examples of inputs, sometimes with correct outputs) and let it find patterns/parameters that generalize to new, unseen data.

Supervised learning: learn a mapping from inputs X to known outputs y . Split further into: - **Regression** — predicting a continuous number (house price, temperature). - **Classification** — predicting a discrete category (spam/not spam, digit 0–9).

Unsupervised learning: find structure in data with no labeled outputs — clustering (group similar points), dimensionality reduction (compress features while preserving structure).

Reinforcement learning: an agent learns to take actions in an environment to maximize cumulative reward, through trial and error and delayed feedback, rather than from a fixed labeled dataset.

The train/validation/test split, and why it exists: - **Training set:** the data the model's parameters are actually fit on. - **Validation set:** used to tune hyperparameters and choose between models — never used to fit parameters directly. - **Test set:** touched exactly once, at the very end, to get an honest, unbiased estimate of how the model will perform on genuinely new data.

If you use the test set to make *any* decision along the way (which model to pick, which hyperparameters to use), you've “leaked” information into your evaluation, and your final performance estimate will be optimistically biased — this is one of the most common beginner mistakes and a frequent root cause of “it worked great in my notebook but failed in production.”

Overfitting vs underfitting — connecting back to the bias-variance tradeoff: A model that's too simple can't capture the true underlying pattern (**underfitting** / high bias) — it performs poorly on both training and validation data. A model that's too complex memorizes the training data, including its noise (**overfitting** / high variance) — it performs great on training data but poorly on validation/test data. The gap between training and validation performance is your primary diagnostic signal for which regime you're in.

5.7 Practice

1. By hand, differentiate $f(x) = (2x^2 + 1)^3$ using the chain rule.

2. Run three iterations of gradient descent by hand on $L(w) = (w+2)^2$, starting at $w = 5$, learning rate $\alpha = 0.2$. Where is it converging, and why does that make sense given the function?
3. Given the numbers $[2, 4, 4, 4, 5, 5, 7, 9]$, compute the mean, median, variance, and standard deviation by hand.
4. In your own words (no notes), explain why a p-value is not “the probability the null hypothesis is true.” Then explain the same distinction for a confidence interval.

6 Week 5 — Regression

6.1 5.1 Linear Regression — Building It From First Principles

The model: predict a continuous target as a weighted sum of features plus a bias term:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w^Tx + b$$

The loss function — Mean Squared Error (MSE):

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

We square the error for two reasons: it makes all errors positive (so they don't cancel out), and it penalizes large errors disproportionately more than small ones — a prediction off by 10 contributes 100x more loss than one off by 1, which pushes the model to avoid large mistakes.

Solution method 1 — the Normal Equation (closed form):

$$\hat{w} = (X^TX)^{-1}X^Ty$$

This comes from setting the gradient of the MSE loss to zero and solving directly. It gives the exact optimal weights in one shot — no iteration needed — but requires inverting X^TX , which is computationally expensive ($O(n^3)$) for a large number of features, and undefined if X^TX is singular (e.g., due to perfectly correlated features).

Solution method 2 — Gradient Descent (iterative): Derived in Week 4: $\frac{\partial L}{\partial w} = \frac{2}{n}X^T(\hat{y} - y)$. Update repeatedly: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$. This scales to huge datasets where the normal equation is impractical, and is the approach used for essentially every larger ML model (since most don't even have a closed-form solution at all).

Python from scratch:

```
import numpy as np

def fit_linear_regression(X, y, lr=0.01, epochs=1000):
    n, d = X.shape
    w = np.zeros(d)
    b = 0
    for _ in range(epochs):
        y_pred = X @ w + b
        error = y_pred - y
        dw = (2/n) * X.T @ error
        db = (2/n) * np.sum(error)
        w -= lr * dw
        b -= lr * db
    return w, b
```

6.2 5.2 Gradient Descent Variants

- **Batch GD:** uses the *entire* dataset to compute each gradient step — stable, smooth convergence, but slow/expensive per step on large datasets.
- **Stochastic Gradient Descent (SGD):** uses *one* random sample per step — very fast per step, but noisy (the loss doesn't decrease smoothly), and that noise can actually help escape shallow local minima or saddle points.
- **Mini-batch GD:** uses a small batch (e.g., 32–256 samples) per step — the practical standard, balancing computational efficiency (vectorized operations on a batch are efficient) with a smoother, more stable descent than pure SGD.

6.3 5.3 Logistic Regression

Despite the name, this is a **classification** algorithm. Instead of predicting a raw number, it predicts a *probability* that an example belongs to the positive class.

The sigmoid function squashes any real number into $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The model computes $z = w^T x + b$ (same linear form as before), then $\hat{y} = \sigma(z) = P(y = 1|x)$.

Why not use MSE here? Plugging a sigmoid into MSE produces a loss surface with many local minima (non-convex) — gradient descent isn't guaranteed to find the global minimum. Instead, logistic regression uses **binary cross-entropy (log loss)**:

$$L = -\frac{1}{n} \sum_i [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

For a true label $y_i = 1$: the loss reduces to $-\log(\hat{y}_i)$ — as $\hat{y}_i \rightarrow 1$ (correct, confident), loss $\rightarrow 0$; as $\hat{y}_i \rightarrow 0$ (wrong, confident), loss $\rightarrow \infty$. This heavily penalizes confident wrong predictions, which is exactly the behavior you want from a probability-calibrated classifier. This loss function is also convex for logistic regression, guaranteeing gradient descent converges to the global minimum.

Decision boundary: classify as positive if $\hat{y} > 0.5$ (equivalently, $z > 0$) — this threshold can be adjusted depending on whether false positives or false negatives are more costly for your problem.

Python:

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def fit_logistic_regression(X, y, lr=0.1, epochs=1000):
    n, d = X.shape
    w = np.zeros(d)
    b = 0
    for _ in range(epochs):
```

```

z = X @ w + b
y_pred = sigmoid(z)
error = y_pred - y
dw = (1/n) * X.T @ error
db = (1/n) * np.sum(error)
w -= lr * dw
b -= lr * db
return w, b

```

6.4 5.4 Regularization

Regularization adds a penalty term to the loss to discourage overly large weights, which reduces overfitting.

Ridge (L2):

$$L = \text{MSE} + \lambda \sum_i w_i^2$$

Shrinks all weights smoothly toward zero, proportionally to their size — large weights get penalized more, but rarely hit exactly zero. Especially useful when features are correlated (multicollinearity), since it stabilizes the solution (makes $X^T X + \lambda I$ always invertible).

Lasso (L1):

$$L = \text{MSE} + \lambda \sum_i |w_i|$$

Can shrink weights all the way to exactly zero — effectively performing automatic feature selection by “turning off” irrelevant features entirely.

Why L1 zeros out weights but L2 doesn’t (geometric intuition): picture the loss contours (ellipses) overlaid on the constraint region defined by the penalty. L1’s constraint region (in 2D) is a diamond with sharp corners sitting exactly on the axes; the loss contours are geometrically likely to first touch this region precisely at a corner, where one weight is exactly zero. L2’s constraint region is a smooth circle with no corners, so the point of contact almost never lands exactly on an axis.

ElasticNet: a weighted combination, $\lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$ — gives some feature selection (from the L1 term) while retaining L2’s stability with correlated features.

λ is a hyperparameter controlling regularization strength — chosen via cross-validation on the validation set, not the training set (a larger λ increases bias but decreases variance).

6.5 Practice

1. Derive the gradient of MSE with respect to b (not just w) for linear regression.
2. Implement linear regression from scratch on a small synthetic dataset (e.g., $y = 3x + 5 + \text{noise}$) and confirm gradient descent recovers weights close to 3 and 5.

3. Explain, in your own words, why using MSE with a sigmoid output creates a non-convex loss surface, while cross-entropy doesn't.
4. Train a Ridge and a Lasso model (using scikit-learn) on the same dataset with several correlated features, print the learned coefficients for both, and observe which one zeroes some out.

7 Week 6 — Classification Algorithms

7.1 6.1 K-Nearest Neighbors (KNN)

The idea: to classify a new point, look at the k closest points in the training data (by some distance metric) and take a majority vote of their labels (for regression, average their values).

Distance metrics: - Euclidean: $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$ — straight-line distance, the default choice. - Manhattan: $d(x, y) = \sum_i |x_i - y_i|$ — sum of absolute differences, useful when movement is grid-like or when you want less sensitivity to large single-feature differences.

Why scaling matters enormously for KNN: if one feature ranges 0–1 and another ranges 0–100,000, the second feature will completely dominate the distance calculation regardless of its actual relevance — always standardize features before using KNN.

Choosing k : small k (e.g., $k = 1$) is very sensitive to noise (high variance, low bias — can overfit to individual noisy points). Large k smooths the decision boundary but can blur genuinely distinct regions (higher bias). Choose k via cross-validation, typically trying odd values (to avoid ties in binary classification).

The curse of dimensionality: as the number of features grows, the volume of space grows exponentially, and points become nearly equidistant from each other — the notion of “nearest” neighbor becomes far less meaningful. This is why KNN performs poorly on high-dimensional data (e.g., raw pixel data) without dimensionality reduction first.

Computational cost: KNN has no real “training” phase (it just stores the data) — all the cost is at prediction time, where it must compute distances to every training point. This makes it slow at inference for large datasets, unlike most other algorithms which are slow to train but fast to predict.

7.2 6.2 Naive Bayes

Applies Bayes’ theorem (Week 3) to classification, with a simplifying (“naive”) assumption: given the class, all features are conditionally independent of each other.

$$P(y|x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

You classify by picking the y that maximizes this product. Despite the independence assumption almost never being literally true (word occurrences in a document, for example, are clearly not independent), Naive Bayes works remarkably well in practice, especially for text classification, because it only needs to rank classes correctly relative to each other — it doesn’t need the individual probability *estimates* to be perfectly accurate, just the final ranking.

Common variant — Multinomial Naive Bayes for text: models $P(x_i|y)$ as word frequencies within each class, making it the classic baseline for spam detection and document classification.

7.3 6.3 Decision Trees

How a tree is built: starting from the root (all data), the algorithm picks the feature and threshold that best splits the data into purer subsets, then recurses on each subset, continuing until a stopping condition (max depth, minimum samples per leaf, or perfect purity).

Measuring “purity” of a split — Entropy:

$$H(S) = - \sum_i p_i \log_2 p_i$$

where p_i is the proportion of class i in set S . Entropy is 0 when a set contains only one class (perfectly pure) and maximal when classes are evenly mixed. **Information gain** from a split is the reduction in entropy: $\text{Gain} = H(\text{parent}) - \sum \frac{|\text{child}|}{|\text{parent}|} H(\text{child})$ — the algorithm picks the split maximizing this gain.

Measuring purity — Gini impurity (alternative, computationally cheaper):

$$\text{Gini}(S) = 1 - \sum_i p_i^2$$

Also 0 for a pure set. In practice, Gini and entropy usually produce very similar trees; Gini is the scikit-learn default because it’s slightly faster to compute (no logarithm).

Worked example: a node has 10 samples: 6 class A, 4 class B. $p_A = 0.6, p_B = 0.4$. $\text{Gini} = 1 - (0.6^2 + 0.4^2) = 1 - (0.36 + 0.16) = 0.48$. $\text{Entropy} = -(0.6 \log_2 0.6 + 0.4 \log_2 0.4) \approx -(0.6 \times (-0.737) + 0.4 \times (-1.322)) \approx 0.971$.

Why trees overfit easily, and how to control it: a fully grown tree can create a leaf for every single training point (perfectly memorizing the training data), which generalizes terribly. Control via: `max_depth`, `min_samples_split`, `min_samples_leaf` (stop splitting if a leaf would have too few samples), or **pruning** (grow the full tree, then remove branches that don’t improve validation performance).

Why trees are valuable beyond raw accuracy: they’re directly interpretable (you can trace the exact if-then path to any prediction) and require no feature scaling (splits are based on thresholds per feature, independent of other features’ scales).

7.4 6.4 Support Vector Machines (SVM)

The core idea: among all hyperplanes that separate two classes, find the one that maximizes the **margin** — the distance from the hyperplane to the nearest points of each class. These nearest points are called **support vectors**, and they’re the only points that actually determine the boundary — all other points could be removed without changing it.

Why maximizing margin matters: intuitively, a decision boundary that sits as far as possible from both classes is more robust to small perturbations in new data, generalizing better than a boundary that just barely separates the training points.

The kernel trick — handling non-linear boundaries: many real datasets aren't linearly separable. Rather than explicitly transforming data into a higher-dimensional space (which could be computationally infeasible for high/infinite dimensions), kernel functions compute the dot product *as if* the data had been transformed, without ever doing the transformation explicitly. Common kernels: - **Linear kernel:** $K(x, x') = x^T x'$ — no transformation, for linearly separable data. - **RBF (Gaussian) kernel:** $K(x, x') = e^{-\gamma \|x - x'\|^2}$ — effectively infinite-dimensional, very flexible, the most commonly used non-linear kernel. - **Polynomial kernel:** $K(x, x') = (x^T x' + c)^d$ — models polynomial decision boundaries.

Hyperparameters: C controls the tradeoff between maximizing the margin and minimizing training misclassifications (small C = wider margin, more tolerance for misclassified points, less overfitting; large C = narrower margin, fits training data more tightly, more overfitting risk). gamma (for RBF) controls how far the influence of a single training point reaches (small gamma = far-reaching, smoother boundary; large gamma = only nearby points matter, more complex/wiggly boundary, higher overfitting risk).

7.5 6.5 Evaluation Metrics for Classification

Confusion matrix: the foundation for every classification metric —

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- **Accuracy** = $\frac{TP+TN}{TP+TN+FP+FN}$ — fraction correct overall. Misleading on imbalanced data.
- **Precision** = $\frac{TP}{TP+FP}$ — of everything you predicted positive, how much was actually right. High precision = few false alarms.
- **Recall (Sensitivity)** = $\frac{TP}{TP+FN}$ — of everything actually positive, how much did you catch. High recall = few missed positives.
- **F1 score** = $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ — the harmonic mean, penalizing extreme imbalance between precision and recall more than a simple average would.

Choosing which matters more, with concrete scenarios: - Spam detection: false positives (blocking a real email) are often more annoying than false negatives (a spam email slipping through) — lean toward precision. - Cancer screening: false negatives (missing a real case) are far more costly than false positives (an unnecessary follow-up test) — lean toward recall.

ROC curve and AUC: the ROC curve plots True Positive Rate (recall) against False Positive Rate ($\frac{FP}{FP+TN}$) as you vary the classification threshold from 0 to 1. AUC (area under this curve) summarizes performance across *all* thresholds into a single number

— 0.5 means random guessing, 1.0 means perfect separation. On severely imbalanced datasets, a **Precision-Recall curve** (and its AUC) is often more informative than ROC-AUC, because ROC-AUC can look deceptively good even when precision is poor, due to the large number of true negatives inflating it.

7.6 Practice

1. Implement KNN from scratch (just Euclidean distance + majority vote) on a small 2D synthetic dataset, and visualize how the decision boundary changes as you vary k from 1 to 15.
2. By hand, compute Gini impurity and entropy for a node with 8 samples: 7 of class A, 1 of class B. Compare to the worked example above — which node is “purer,” and does that match your intuition?
3. Train an SVM with an RBF kernel on a non-linearly-separable 2D dataset (e.g., scikit-learn’s `make_moons`), and plot the decision boundary at three different `gamma` values to see the effect directly.
4. For a binary classifier outputting probabilities, compute precision, recall, and F1 at threshold 0.5, then recompute at threshold 0.3 — explain the direction each metric moves and why.

8 Week 7 — Ensemble Methods

8.1 7.1 Why Combine Models At All

A single model has a fixed bias-variance profile. Ensembles combine multiple models to get something better than any individual one could achieve alone — but *why* this works depends entirely on *how* you combine them, which is the key distinction between bagging and boosting.

8.2 7.2 Bagging (Bootstrap Aggregating)

The mechanism: 1. Create many **bootstrap samples** — random samples of the same size as the original dataset, drawn *with replacement* (so some points appear multiple times, others not at all; on average about 63% of unique points appear in each sample). 2. Train an independent model on each bootstrap sample. 3. Combine predictions: majority vote for classification, average for regression.

Why this reduces variance: each model, trained on a slightly different sample, makes somewhat different errors — some overestimate, some underestimate, in ways that are only weakly correlated across models. When you average many weakly-correlated errors, they partially cancel out, shrinking the ensemble’s overall variance, while each individual model’s bias (which comes from the model class itself, not the specific sample) stays roughly the same. This is why bagging is most effective with high-variance, low-bias base models — like deep, unpruned decision trees.

8.3 7.3 Random Forest

Random Forest is bagging applied specifically to decision trees, with one crucial addition: at each split, only a **random subset of features** is considered (not all of them). This further **decorrelates** the trees — without this, if one feature is very strong, nearly every bootstrapped tree would split on it first regardless of the sample, making the trees highly correlated with each other and reducing the variance-cancelling benefit of averaging.

Key hyperparameters: - `n_estimators` — number of trees; more is generally better (up to diminishing returns) and doesn’t increase overfitting risk the way it would for boosting. - `max_features` — size of the random feature subset per split (commonly $\sqrt{n_features}$ for classification). - `max_depth`, `min_samples_leaf` — control individual tree complexity.

Feature importance: Random Forest naturally provides feature importance scores (based on how much each feature reduces impurity across all trees, weighted by how many samples pass through each split) — a genuinely useful, low-effort tool for understanding what’s driving predictions, though it can be biased toward high-cardinality features.

8.4 7.4 Boosting

The mechanism (general idea): train models **sequentially**, where each new model is trained specifically to correct the mistakes of the ensemble built so far, rather than

being trained independently.

AdaBoost (Adaptive Boosting): after each weak learner (often a shallow “decision stump”) is trained, misclassified samples get their weights *increased*, so the next weak learner is forced to pay more attention to the examples the ensemble is currently getting wrong. Final prediction is a weighted vote of all weak learners, where better-performing learners get more say.

Gradient Boosting — the general framework: instead of reweighting samples, each new model is trained to predict the **residual errors** (for regression with squared loss, this is literally $y - \hat{y}_{\text{so far}}$) of the current ensemble. More generally, each new model fits the *negative gradient* of the loss function with respect to the current predictions — which is why it’s called “gradient” boosting: you’re performing gradient descent, but in the space of *functions* (models) rather than in the space of parameters.

Worked intuition: suppose the true value is 100, and your first tree predicts 80. The residual is 20. Your second tree is trained to predict that 20 (not 100) — if it predicts 15, your ensemble now predicts $80 + 15 = 95$, with a remaining residual of 5. A third tree targets that 5, and so on — each tree chips away at what’s still wrong, progressively reducing bias.

Why boosting reduces bias (and can reduce variance too, if regularized well): each new model explicitly targets the ensemble’s current errors, so the ensemble’s *fit* to the training data keeps improving — directly attacking bias, which bagging doesn’t do (bagging’s average bias is the same as a single tree’s bias). The risk: if you boost for too many rounds without regularization, the ensemble can start fitting noise in the residuals, increasing variance and overfitting — this is why boosting needs careful tuning (learning rate, number of rounds, early stopping), whereas bagging is comparatively “hard to overfit” by adding more trees.

8.5 7.5 XGBoost, LightGBM, CatBoost

These are production-grade, highly optimized gradient boosting implementations, each with a distinguishing idea:

XGBoost: - Adds an explicit **regularization term** (both L1 and L2 on leaf weights, plus a penalty on the number of leaves) directly into the objective function being optimized — this is boosting with overfitting control built into the math itself, not just an external hyperparameter. - Uses **second-order gradient information** (both the gradient and the Hessian/second derivative of the loss) via a Newton’s-method-style approximation, which lets it choose better splits more efficiently than using gradient information alone.

LightGBM: - Uses **histogram-based** split-finding: instead of considering every possible threshold for a continuous feature (expensive), it buckets values into discrete histogram bins first, dramatically speeding up split evaluation on large datasets. - Grows trees **leaf-wise** (always splitting the leaf that will reduce loss the most next) rather than **level-wise** (splitting every leaf at the current depth before going deeper, as XGBoost traditionally does) — leaf-wise growth can reach lower loss with fewer splits, but is more prone to overfitting on small datasets, so it needs `max_depth/num_leaves` limits.

CatBoost: - Handles **categorical features natively**, without needing manual one-hot/target encoding, using a specialized statistics-based encoding computed in a way that avoids leaking target information. - Uses **ordered boosting** — a technique that computes residuals for each training example using only a model trained on data that came “before” it (in a randomly permuted order), specifically to reduce target leakage/overfitting that can occur in standard gradient boosting’s residual computation.

Practical guidance on choosing: XGBoost is the most battle-tested, general default. LightGBM is often faster and more memory-efficient on very large datasets. CatBoost is the strongest choice when you have many categorical features and want to avoid manual encoding work — all three usually land within a similar accuracy range on tabular data when properly tuned, so speed, categorical handling, and ecosystem familiarity often matter more than raw accuracy differences.

8.6 7.6 Hyperparameter Tuning

- **Grid Search:** exhaustively tries every combination of a specified hyperparameter grid — thorough but computationally expensive, scaling exponentially with the number of hyperparameters.
- **Random Search:** samples random combinations instead — often finds a comparably good configuration far faster than grid search, because typically only a few hyperparameters actually matter much, and random search explores their range more efficiently than a fixed grid.
- **Bayesian optimization (e.g., Optuna):** builds a probabilistic model of how hyperparameters affect performance, using past trial results to intelligently choose the *next* configuration to try (favoring promising regions of the search space) — generally more sample-efficient than both grid and random search, especially valuable when each training run is expensive.

Cross-validation, tying it together: rather than a single train/validation split (which can give a noisy performance estimate depending on which points happened to land in validation), **k-fold cross-validation** splits the training data into k folds, trains on $k - 1$ folds and validates on the remaining one, rotating through all k folds, then averages the results — giving a more robust estimate of how a given hyperparameter configuration will generalize, at the cost of $k \times$ the compute.

8.7 Practice

1. Train a single decision tree and a Random Forest (same `max_depth`) on the same dataset; compare train vs validation accuracy for both, and explain the gap you observe in terms of bias/variance.
2. Manually trace three rounds of gradient boosting on a toy regression example (5 data points), computing residuals after each round, to build direct intuition before touching a library.
3. Train an XGBoost model and print its feature importances; compare them to a Random Forest’s feature importances on the same data — are the top features the same?
4. Run Grid Search and Random Search (same total number of trials) for tuning a Random Forest’s `n_estimators`, `max_depth`, and `min_samples_leaf`; compare

best validation scores and total runtime.

9 Week 8 — Unsupervised Learning, Feature Engineering, and Project 1

9.1 8.1 K-Means Clustering

The algorithm, step by step: 1. Choose the number of clusters k , and initialize k centroids (often randomly, or better, via **k-means++**, which spreads initial centroids apart to reduce the chance of poor local optima). 2. **Assignment step:** assign each point to its nearest centroid (by Euclidean distance). 3. **Update step:** recompute each centroid as the mean of all points currently assigned to it. 4. Repeat steps 2–3 until assignments stop changing (convergence).

What it's actually optimizing: K-Means minimizes **inertia** — the sum of squared distances from each point to its assigned centroid, $\sum_i \|x_i - \mu_{c_i}\|^2$. Each iteration provably decreases (or maintains) this objective, guaranteeing convergence — but only to a *local* minimum, which is why initialization matters and why it's common practice to run K-Means several times with different initializations and keep the best result.

Choosing k : - **Elbow method:** plot inertia against k ; inertia always decreases as k increases, but look for the “elbow” point where the rate of decrease sharply slows — beyond that point, adding more clusters gives diminishing returns. - **Silhouette score:** measures how similar a point is to its own cluster versus the next-nearest cluster, ranging from -1 to 1 (higher is better); average silhouette score across all points gives a single number to compare different k values.

Limitations: assumes roughly spherical, similarly-sized clusters (because it's built on Euclidean distance to a single centroid) — struggles badly with elongated, unevenly sized, or non-convex cluster shapes. Also requires specifying k in advance.

9.2 8.2 Hierarchical Clustering

Agglomerative (bottom-up) approach: start with every point as its own cluster, then repeatedly merge the two closest clusters, until everything is in one cluster. The record of merges forms a **dendrogram** — a tree diagram showing the order and distance at which clusters merged. You can then “cut” the dendrogram at any height to get a specific number of clusters, without needing to decide k in advance.

Linkage criteria (how “distance between clusters” is defined): - **Single linkage:** distance between the closest pair of points across two clusters — can produce elongated, “chained” clusters. - **Complete linkage:** distance between the farthest pair of points — tends to produce more compact, evenly-sized clusters. - **Average linkage:** average distance between all cross-cluster pairs — a middle ground. - **Ward's method:** merges the pair of clusters that leads to the smallest increase in total within-cluster variance — generally produces the most balanced, interpretable clusters and is the most common default.

9.3 8.3 DBSCAN (Density-Based Clustering)

The idea: clusters are regions of high point density, separated by regions of low density. Two parameters: `eps` (the neighborhood radius) and `min_samples` (minimum points required within that radius for a point to be considered a “core point”).

A point is a **core point** if at least `min_samples` other points lie within `eps` of it. A **border point** is within `eps` of a core point but doesn’t itself have enough neighbors. A **noise point** belongs to neither — DBSCAN naturally identifies outliers, unlike K-Means, which forces every point into some cluster.

Key advantages over K-Means: doesn’t require specifying the number of clusters in advance, and can find arbitrarily shaped (non-spherical) clusters, since it’s based on connectivity through dense regions rather than distance to a single centroid.

9.4 8.4 PCA — Full Walkthrough

Goal: find a lower-dimensional representation of the data that preserves as much variance (information) as possible.

Step by step: 1. **Mean-center the data:** subtract each feature’s mean, so the data is centered at the origin (PCA requires this — otherwise the “directions of maximum variance” would be skewed by the data’s location, not its actual spread). 2. **Compute the covariance matrix:** $\Sigma = \frac{1}{n} X^T X$ (using the mean-centered X). 3. **Compute eigenvectors and eigenvalues of Σ .** Each eigenvector is a candidate new axis (“principal component”); its eigenvalue tells you how much variance lies along it. 4. **Sort eigenvectors by eigenvalue, descending.** The first principal component is the direction of maximum variance in the data; the second is the direction of maximum *remaining* variance, orthogonal to the first, and so on. 5. **Project the data onto the top k eigenvectors** to get your reduced-dimensional representation.

How to choose k : plot the cumulative **explained variance ratio** (each eigenvalue divided by the sum of all eigenvalues, summed cumulatively) against the number of components, and pick k where it crosses a threshold you’re comfortable with (commonly 90–95%).

Important caveat — always scale features first: since PCA is driven by variance, a feature measured in larger units (e.g., salary in dollars vs. age in years) will dominate the principal components purely due to its scale, not its actual importance — always standardize features before applying PCA.

9.5 8.5 t-SNE and UMAP (Visualization-Focused Dimensionality Reduction)

Unlike PCA (which preserves global variance structure via a *linear* projection), **t-SNE** and **UMAP** are non-linear techniques that focus on preserving *local* neighborhood structure — points that were close in high-dimensional space stay close in the low-dimensional (typically 2D) embedding, even if that requires “tearing apart” the global geometry.

Important limitation: distances *between* well-separated clusters in a t-SNE/UMAP plot aren't reliably meaningful (don't read too much into "cluster A is closer to B than to C" from the plot alone), and — crucially — there's no simple way to map a *new, unseen* point into an existing t-SNE embedding, which is why these methods are used almost exclusively for visualization/exploration rather than as a preprocessing step before a downstream supervised model (where PCA is used instead).

9.6 8.6 Feature Engineering

Scaling: - **Standardization (Z-score):** $x' = \frac{x-\mu}{\sigma}$ — resulting feature has mean 0, standard deviation 1. Required for distance-based methods (KNN, SVM, K-Means) and gradient-descent-based methods (linear/logistic regression, neural networks), since these are all sensitive to the relative scale of features. - **Min-Max scaling:** $x' = \frac{x-\min}{\max-\min}$ — rescales to a fixed range, typically [0,1]. Sensitive to outliers (a single extreme value compresses everything else into a tiny range). - **Tree-based models (decision trees, random forests, gradient boosting) don't need scaling** — splits are based on per-feature thresholds, unaffected by the scale of other features or even of the same feature (a monotonic rescaling doesn't change which points fall on which side of a threshold).

Encoding categorical variables: - **One-hot encoding:** create a binary column per category — appropriate for nominal (unordered) categories with relatively low cardinality; avoids implying a false numeric order. - **Ordinal encoding:** map categories to integers reflecting a genuine order (e.g., "low"→0, "medium"→1, "high"→2) — only appropriate when the order is real and meaningful. - **Target/mean encoding:** replace a category with the average target value for that category — powerful for high-cardinality categorical features (e.g., zip code, product ID) where one-hot encoding would create too many columns, but risks **target leakage** if not done carefully (must be computed using cross-validation folds, never using the full dataset including the row being encoded, or your model will implicitly "see" the label).

Handling missing data: - **Deletion:** drop rows/columns with missing values — only reasonable when missingness is minimal and appears random (not systematically related to the target). - **Imputation:** fill with mean/median (robust to outliers, use median) for numeric features, mode for categorical; more sophisticated options include KNN imputation (fill based on similar rows) or iterative imputation (model each feature with missing values as a function of the other features). - **Missingness indicator:** sometimes *whether* a value is missing is itself informative (e.g., a survey question left blank might correlate with the target) — adding a binary "was this missing" column alongside the imputed value can capture this signal.

Handling outliers: - **Capping/winsorizing:** clip extreme values to a percentile boundary (e.g., cap anything above the 99th percentile) rather than removing them entirely. - **Log transformation:** for right-skewed data (e.g., income, prices), taking $\log(x)$ compresses large values and often makes the distribution closer to normal, which helps many models (and is a near-default step for monetary features).

9.7 8.7 Project 1 — End-to-End ML Project (Guided Structure)

This week's project should demonstrate the entire pipeline, start to finish, and become a genuine portfolio piece:

1. **Pick a real, moderately messy dataset** (Kaggle is a good source) — avoid pre-cleaned toy datasets; messiness is the point.
2. **EDA:** distributions, missing values, correlations, outliers — document what you found and what decisions it led to.
3. **Feature engineering:** scaling, encoding, handling missing values and outliers, guided by what your EDA revealed.
4. **Baseline model first:** always start with something simple (e.g., logistic/linear regression) before reaching for a complex model — this gives you a reference point to know whether added complexity is actually earning its keep.
5. **Model comparison:** try 2–3 model families (e.g., logistic regression, Random Forest, XGBoost), using consistent cross-validation, and compare fairly.
6. **Evaluate thoughtfully:** pick metrics that match the actual problem (not just accuracy by default — revisit Week 6's discussion of precision/recall tradeoffs).
7. **Write a clear README:** problem statement, data source, approach, results, and what you'd try next with more time — this narrative is what makes a project useful in an interview, not just the code.

9.8 Practice

1. Implement K-Means from scratch (assignment + update steps only, no library) on a small 2D synthetic dataset with 3 visually separated clusters, and confirm it converges to sensible centroids.
2. Run PCA on a dataset with 10+ features, plot cumulative explained variance, and decide how many components you'd keep to retain 95% of the variance.
3. On a dataset with a high-cardinality categorical column, implement target encoding using proper cross-validation folds (not the naive full-dataset version), and explain in one paragraph why the naive version leaks information.
4. Complete Project 1 following the structure in 8.7, and push it to GitHub with a README a stranger could understand in two minutes.

10 Week 9 — Neural Network Fundamentals

10.1 9.1 From a Single Neuron to a Network

A **single neuron (perceptron)** computes a weighted sum of its inputs, adds a bias, and applies a non-linear activation function:

$$z = w^T x + b, \quad a = f(z)$$

This is structurally identical to logistic regression — in fact, logistic regression *is* a single neuron with a sigmoid activation. The power of neural networks comes from stacking many neurons into **layers**, and stacking many layers, so the network can compose simple transformations into extremely complex ones.

A **layer** is a set of neurons, each producing one output, collectively transforming an input vector into an output vector:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

where l indexes the layer, $W^{(l)}$ is a weight *matrix* (each row is one neuron's weights), and $a^{(l-1)}$ is the previous layer's output (activation).

A **Multi-Layer Perceptron (MLP)** stacks several such layers: input layer $\rightarrow$ one or more hidden layers $\rightarrow$ output layer. Each hidden layer's job is to learn increasingly useful, more abstract transformations of the raw input, so that by the time data reaches the output layer, the problem has become linearly separable (or close to it) even if it wasn't in the raw input space.

10.2 9.2 Why Non-Linear Activations Are Non-Negotiable

If every layer used a purely linear (identity) activation, the entire network — no matter how many layers — would mathematically collapse into a single linear transformation (because the composition of linear functions is itself linear). A 50-layer “deep” network without non-linearities would have exactly the same representational power as ordinary linear regression. Non-linear activation functions are what let the network approximate genuinely complex, curved decision boundaries and functions.

10.3 9.3 Activation Functions in Depth

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$, output range $(0, 1)$. Derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, which has a maximum value of 0.25 at $z = 0$ and approaches 0 as $|z|$ grows large — this is the **vanishing gradient** problem: for large positive or negative inputs, the gradient flowing backward through a sigmoid is nearly zero, so earlier layers in a deep network barely get updated during backpropagation.

Tanh: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, output range $(-1, 1)$, zero-centered (unlike sigmoid). Zero-centering helps gradient descent converge somewhat more efficiently, but tanh still saturates for large $|z|$ and suffers the same vanishing gradient issue, just less severely than sigmoid.

ReLU (Rectified Linear Unit): $f(z) = \max(0, z)$. Derivative: 1 for $z > 0$, 0 for $z < 0$ (undefined exactly at 0, conventionally set to 0 or 1). Because the gradient is a constant 1 for all positive inputs (no saturation, no shrinking), ReLU largely avoids the vanishing gradient problem for active neurons, and it's extremely cheap to compute — this is why it's the default choice for hidden layers in modern deep networks. **The “dying ReLU” problem:** if a neuron's weights update such that it always outputs a negative z (and hence 0) for every input in the training set, its gradient becomes permanently 0, and it can never recover (“it's dead”) — this can happen with a poor weight initialization or too large a learning rate. **Leaky ReLU:** $f(z) = z$ if $z > 0$, else αz for a small constant α (e.g., 0.01) — allows a small non-zero gradient when $z < 0$, preventing neurons from fully dying.

Softmax (output layer, multi-class classification): $\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$ — converts a vector of raw scores into a valid probability distribution (all values in $(0, 1)$, summing to 1). Paired with **categorical cross-entropy loss** for multi-class problems, the same way sigmoid pairs with binary cross-entropy for binary classification.

10.4 9.4 Forward Propagation, Concretely

For a network with one hidden layer, input x , weights $W^{(1)}, W^{(2)}$, biases $b^{(1)}, b^{(2)}$:

$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad a^{(1)} = \text{ReLU}(z^{(1)})$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}, \quad \hat{y} = \text{softmax}(z^{(2)}) \text{ (or sigmoid, for binary output)}$$

This is literally just repeated matrix multiplication + non-linearity, applied layer after layer — nothing more mysterious than that.

10.5 9.5 Backpropagation — The Full Mechanics

The goal: compute $\frac{\partial L}{\partial W^{(l)}}$ and $\frac{\partial L}{\partial b^{(l)}}$ for *every* layer l , so gradient descent can update every weight in the network.

The key insight: because the network is a composition of functions, the chain rule lets you compute these gradients working **backward** from the output, reusing intermediate results at each step — this reuse is what makes backpropagation efficient (computing each layer's gradient independently from scratch would be vastly more expensive).

Define $\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}$ — how much the loss changes with respect to layer l 's pre-activation output. This is the quantity that gets passed backward.

At the output layer (for softmax + cross-entropy, or sigmoid + binary cross-entropy — both simplify beautifully): $\delta^{(\text{output})} = \hat{y} - y$ — the error is simply the difference between predicted and true values. This elegant simplification is exactly *why* those loss/activation pairings are used together.

For a hidden layer, the error is propagated backward from the next layer:

$$\delta^{(l)} = (W^{(l+1)T} \delta^{(l+1)}) \odot f'(z^{(l)})$$

In words: take the next layer’s error, “send it backward” through that layer’s weights (transposed — reversing the forward direction), and multiply element-wise ($\odot$) by the derivative of this layer’s activation function (this is the chain rule, applied one layer at a time).

Once you have $\delta^{(l)}$ for every layer, the weight gradients are:

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} a^{(l-1)T}, \quad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}$$

Why this matters to actually understand (not just call `.backward()`): every debugging question about a network that won’t train — dead neurons, vanishing/exploding gradients, why a particular activation or initialization choice matters — traces directly back to this mechanism. If you understand that $\delta^{(l)}$ depends on $f'(z^{(l)})$, you immediately understand *why* an activation function that saturates (has near-zero derivative) kills learning in earlier layers.

10.6 9.6 Optimizers

SGD with momentum: rather than using the raw gradient at each step, maintain a “velocity” — a running exponential average of past gradients — and use *that* to update weights:

$$v \leftarrow \beta v + (1 - \beta) \nabla L, \quad w \leftarrow w - \alpha v$$

This helps the optimizer power through small local bumps/noise in the loss surface and accelerates convergence in directions where gradients consistently point the same way (like a ball rolling downhill and building up speed).

Adam (Adaptive Moment Estimation): combines momentum (tracking a running average of the gradient — the “first moment”) with an *adaptive, per-parameter learning rate* based on a running average of the squared gradient (the “second moment”):

$$m \leftarrow \beta_1 m + (1 - \beta_1) \nabla L, \quad v \leftarrow \beta_2 v + (1 - \beta_2) (\nabla L)^2$$

$$w \leftarrow w - \alpha \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

(where $\hat{m}, \hat{v}$ are bias-corrected versions of m, v). The effect: parameters with historically large gradients get smaller effective updates (dividing by $\sqrt{\hat{v}}$ dampens them), while parameters with small, infrequent gradients get relatively larger updates — this adaptivity is why Adam generally converges quickly with minimal manual learning-rate tuning, making it the default optimizer for most deep learning tasks.

10.7 9.7 Weight Initialization and Batch Normalization

Why initialization matters: if all weights start at the same value (e.g., all zero), every neuron in a layer computes the exact same output and receives the exact same gradient — they never differentiate from each other, no matter how long you train (“symmetry problem”). Weights must be initialized randomly. But *how* randomly also matters: too large, and activations/gradients can explode through the layers; too

small, and they vanish. - **Xavier/Glorot initialization:** scales initial weights based on the number of input and output units of a layer, designed to keep the variance of activations roughly constant across layers — a good match for sigmoid/tanh activations. - **He initialization:** a variant scaled specifically for ReLU activations (accounting for the fact that ReLU zeros out roughly half its inputs).

Batch Normalization: normalizes the inputs to a layer (across each mini-batch) to have zero mean and unit variance, then applies a learned scale (γ) and shift (β) so the network can undo the normalization if that's actually optimal. This stabilizes training by reducing how much the distribution of each layer's inputs shifts as earlier layers' weights change during training ("internal covariate shift"), which in turn allows higher learning rates and faster, more stable convergence, with a mild regularizing side-effect.

10.8 9.8 PyTorch — Practical Implementation

```
import torch
import torch.nn as nn
import torch.optim as optim

class SimpleNet(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

model = SimpleNet(input_dim=20, hidden_dim=64, output_dim=2)
criterion = nn.CrossEntropyLoss() # combines softmax + cross-entropy internally
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(100):
    optimizer.zero_grad() # reset gradients (they accumulate by default in PyTorch)
    outputs = model(X_train) # forward pass
    loss = criterion(outputs, y_train)
    loss.backward() # backpropagation – computes all gradients automatically
    optimizer.step() # apply the gradient update
```

What `loss.backward()` is actually doing: PyTorch builds a computation graph as you run the forward pass, recording every operation. Calling `.backward()` walks this graph in reverse, applying the chain rule automatically at each recorded operation — this is exactly the backpropagation math from Section 9.5, done for you. Understand-

ing that math is what lets you debug when `.backward()` produces NaN gradients or a model that refuses to learn.

10.9 Practice

1. By hand, run one forward pass and one backward pass of a tiny 2-input, 1-hidden-neuron (ReLU), 1-output network, given specific starting weights, to compute the initial gradients.
2. Implement a 2-layer neural network from scratch in NumPy (forward and backward pass, no PyTorch), and train it on a simple synthetic dataset (e.g., XOR — a classic example that a single linear layer *cannot* solve, but a 2-layer network can).
3. Rebuild the same network in PyTorch and confirm it reaches similar accuracy.
4. Deliberately initialize all weights in a small network to zero, train it, and observe that it fails to learn properly — connect what you see back to the symmetry problem described in 9.7.

11 Week 10 — Convolutional Neural Networks (CNNs)

11.1 10.1 Why Fully-Connected Layers Fail on Images

A small 224×224 RGB image has $224 \times 224 \times 3 \approx 150,000$ input values. A single fully-connected layer with just 1000 neurons would need 150,000,000 weights — computationally infeasible, and prone to severe overfitting given typical dataset sizes. Worse, a fully-connected layer treats every pixel as independent, ignoring the fact that nearby pixels are highly related (spatial structure) and that the same visual pattern (an edge, a texture) can appear anywhere in the image.

11.2 10.2 The Convolution Operation

A **filter (kernel)** — a small matrix of learnable weights (e.g., 3×3) — slides across the input image, and at each position computes a dot product between the filter and the local patch of the image it currently covers, producing one value in the output **feature map**.

Worked example (tiny, for intuition): a 3×3 input patch

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

convolved with a 3×3 edge-detecting filter

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

gives (element-wise multiply, then sum): $(-1 - 2 - 3 + 0 + 0 + 0 + 7 + 8 + 9) = 18$ — this single number is one entry in the output feature map. As this filter slides across a real image, it produces large values where there's a strong horizontal edge and near-zero values in flat/uniform regions — hence “edge detector.”

Key properties that make convolution powerful for images: - **Local connectivity:** each output value only depends on a small local region of the input, matching the intuition that nearby pixels are what's relevant to detecting a local feature. - **Parameter sharing:** the *same* filter (same weights) is applied at every spatial position — a vertical-edge detector useful in the top-left corner is equally useful in the bottom-right, so one filter learns one reusable pattern-detector for the entire image, massively reducing parameters compared to a fully-connected layer.

Stride: how many pixels the filter moves at each step. Stride 1 moves one pixel at a time (dense output); stride 2 skips every other position (halves the output spatial size, reduces computation).

Padding: adding border pixels (usually zeros) around the input before convolving. “Valid” padding (none) shrinks the output; “same” padding adds just enough border so the output spatial size matches the input — useful for building deep networks without the feature map shrinking to nothing after many layers.

Multiple filters, multiple channels: a convolutional layer typically has *many* filters (e.g., 64), each learning to detect a different pattern, producing 64 separate feature maps stacked together as the layer’s output — this stack becomes the “channel” dimension for the next layer.

11.3 10.3 Pooling

Max pooling: slides a window (e.g., 2×2) across the feature map and outputs the *maximum* value in each window, discarding the rest. This (a) reduces the spatial dimensions (downsampling), reducing computation in later layers, and (b) provides a degree of **translation invariance** — if a detected feature shifts slightly within the pooling window, the max output stays the same, so the network becomes somewhat robust to small positional shifts in the input.

Average pooling: takes the average instead of the max within each window — smoother, less commonly used in hidden layers now, but common in the final “global average pooling” layer of modern architectures (replacing large fully-connected layers at the end, reducing parameters and overfitting risk).

11.4 10.4 The Feature Hierarchy — Why Depth Matters

Stacking convolutional layers builds an increasingly abstract hierarchy of learned features: - **Early layers** learn simple, generic patterns: edges, color blobs, simple textures. - **Middle layers** combine these into more complex shapes: corners, textures, simple object parts (an eye-like shape, a wheel-like shape). - **Deep layers** combine those into high-level, task-specific concepts: entire object parts or whole objects (a face, a car). This hierarchy emerges automatically from training — no one hand-designs “detect an eye” — and it’s precisely *why* transfer learning works so well: the early/middle layers’ generic features (edges, textures) are useful for almost any visual task, not just the one the network was originally trained on.

11.5 10.5 Classic Architectures

LeNet (1998): the original CNN, designed for digit recognition (MNIST) — a handful of convolution + pooling layers followed by fully-connected layers. Tiny by today’s standards, but it established the core convolution → pooling → fully-connected pattern still used today.

VGG: demonstrated that stacking many small 3×3 filters (rather than fewer, larger filters like 7×7 or 11×11) achieves a larger effective “receptive field” (the region of the original input that influences a given output value) while using *fewer total parameters* and adding more non-linearities along the way — two stacked 3×3 convolutions have the same receptive field as one 5×5 convolution, but with fewer parameters and an extra activation function in between. Very deep (16-19 weight layers) but computationally heavy and prone to vanishing gradients at that depth without further tricks.

ResNet — solving the “very deep networks are hard to train” problem: researchers found that simply stacking more layers eventually made training *worse*, not

better — even on the *training* set (not just overfitting; genuine optimization difficulty caused by vanishing gradients through many layers). ResNet’s fix: **residual/skip connections**. Instead of a block of layers learning a direct mapping $H(x)$, it learns a residual $F(x) = H(x) - x$, and the block’s actual output is $F(x) + x$ — the input is added directly to the block’s output via a shortcut connection that bypasses the layers entirely.

Why this works, mechanically: during backpropagation, the gradient can flow backward through the identity shortcut path (whose local derivative is exactly 1) in addition to through the convolutional layers. This gives gradients a direct, undiminished route back to earlier layers, largely sidestepping the vanishing gradient problem even in networks over 100 layers deep. It also makes optimization easier in another sense: if the ideal transformation for a block turns out to be close to the identity function (i.e., this block isn’t very useful), it’s much easier for the network to learn $F(x) \approx 0$ (push weights toward zero) than to learn an exact identity mapping from scratch using ordinary weight layers.

11.6 10.6 Transfer Learning

Why train from scratch is often the wrong choice: training a large CNN from scratch requires huge labeled datasets (ImageNet has 1.2 million images) and substantial compute — usually impractical for a typical project or company-specific dataset.

Feature extraction approach: take a model pretrained on ImageNet (e.g., ResNet50), **freeze** all its convolutional layers (don’t update their weights), remove the original final classification layer, and train only a new, small classification head on top, using your own (much smaller) dataset. This works because the frozen layers’ learned features (edges, textures, shapes) are generically useful, and you’re only training the small part that maps those generic features to *your* specific classes.

Fine-tuning approach: after training the new head, unfreeze some or all of the pretrained layers and continue training the *entire* network on your data, but at a much **lower learning rate** than you’d use from scratch — you want to gently adapt the pretrained features to your specific task, not destroy the useful patterns they already encode with large, disruptive updates.

When to choose which: feature extraction is preferable with a small dataset (less risk of overfitting, since you’re training far fewer parameters); fine-tuning is preferable with a moderately larger dataset that’s still much smaller than ImageNet, where adapting the deeper, more task-specific features can meaningfully improve performance.

```
import torchvision.models as models
import torch.nn as nn

model = models.resnet50(pretrained=True)

for param in model.parameters():          # freeze all pretrained layers
    param.requires_grad = False
```

```
model.fc = nn.Linear(model.fc.in_features, num_classes) # replace final layer  
# only model.fc's parameters have requires_grad=True now, so only it gets trained
```

11.7 Practice

1. By hand, convolve a small 4×4 input with a 2×2 filter (stride 1, no padding), and separately compute the output size formula: $\frac{n-f}{s} + 1$ (verify your hand calculation matches).
2. Build a small CNN (2-3 conv layers + pooling + a final fully-connected layer) in PyTorch and train it on a small image dataset (e.g., CIFAR-10 subset), tracking train vs validation accuracy.
3. Apply transfer learning: take a pretrained ResNet, freeze its layers, and fine-tune only the final layer on a small custom image dataset (even 100-200 images per class is enough to see it work) — compare accuracy against the from-scratch CNN from exercise 2.
4. In your own words, explain why a skip connection's derivative being “exactly 1” is the key mechanical reason it helps gradients flow through very deep networks.

12 Week 11 — Sequence Models: RNN, LSTM, GRU

12.1 11.1 Why Sequences Need a Different Architecture

CNNs and standard MLPs assume fixed-size, order-independent (or spatially-local) inputs. Text, speech, time series, and any data where **order matters** and **length varies** need a different approach — one that can process elements one at a time while maintaining some notion of “memory” of what came before.

12.2 11.2 Recurrent Neural Networks (RNNs)

The core idea: process a sequence $x_1, x_2, \dots, x_T$ one element at a time, maintaining a **hidden state** h_t that summarizes everything seen so far, updated at each step by combining the current input with the previous hidden state:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = W_{hy}h_t + b_y \quad (\text{if an output is needed at this step})$$

Crucially, the *same* weight matrices (W_{hh}, W_{xh}) are reused at every time step — this is parameter sharing across time, analogous to how CNNs share filter weights across spatial positions, and it’s what lets an RNN handle sequences of any length with a fixed number of parameters.

Unrolling through time: conceptually, you can “unroll” an RNN into a long chain — one copy of the network per time step, each feeding its hidden state to the next — which is a useful mental model for understanding both forward propagation and, especially, backpropagation through this structure.

12.3 11.3 Backpropagation Through Time (BPTT) and the Vanishing Gradient Problem

To train an RNN, gradients must flow backward not just through layers (as in a standard network) but backward through *every time step* of the unrolled sequence. Because the same weight matrix W_{hh} is applied repeatedly, computing the gradient with respect to an early time step’s hidden state involves multiplying by W_{hh} (and the activation’s derivative) once per time step in between:

$$\frac{\partial h_T}{\partial h_1} = \prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}}$$

If the terms in this product are consistently less than 1 in magnitude (which happens easily, since $\tanh'$ is at most 1 and often much smaller), the product shrinks exponentially as the sequence gets longer — gradients **vanish**, and the network effectively can’t learn dependencies spanning more than a handful of time steps. (If the terms are consistently greater than 1, gradients can instead **explode** — growing exponentially, causing unstable, huge updates; this is typically handled with **gradient clipping**, capping the gradient’s norm at a maximum value.)

The practical consequence: a vanilla RNN struggles badly to learn relationships between, say, a word at the start of a long sentence and a word at the end — exactly the kind of long-range dependency that language understanding often requires.

12.4 11.4 LSTM (Long Short-Term Memory)

LSTMs were specifically designed to fix the vanishing gradient problem by introducing a separate **cell state** c_t — a pathway that information can flow along with only *minor, controlled, additive* modifications at each step, rather than being repeatedly squashed through a non-linearity and matrix multiplication like the RNN’s hidden state.

The three gates (each a sigmoid-activated layer, outputting values between 0 and 1 that act as “how much to let through”):

Forget gate — decides what fraction of the old cell state to keep:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input gate — decides what new information to add to the cell state:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

Cell state update — combines the forget and input gates:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Notice this update is **additive** ($f_t \odot c_{t-1}$ plus $i_t \odot \tilde{c}_t$), not a repeated multiplication through a squashing non-linearity like the vanilla RNN’s hidden state update — this additive structure is the crucial difference that lets gradients flow backward through the cell state across many time steps with far less shrinkage.

Output gate — decides what part of the cell state becomes the visible hidden state:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t)$$

Intuition for the gates in plain language: the forget gate is “how much of my old memory is still relevant”; the input gate is “how much of this new information is worth remembering”; the output gate is “how much of my memory should I actually reveal/use right now for this step’s output.” All three are *learned* — the network figures out through training what’s worth remembering and forgetting for the task at hand.

12.5 11.5 GRU (Gated Recurrent Unit)

A simplified alternative to LSTM, with two gates instead of three, and no separate cell state (the hidden state itself carries the “memory”):

Update gate z_t — controls how much of the previous hidden state to keep versus replace with new content (essentially combines the roles of LSTM’s forget and input gates into one).

Reset gate r_t — controls how much of the previous hidden state to use when computing the candidate new content.

$$z_t = \sigma(W_z[h_{t-1}, x_t]), \quad r_t = \sigma(W_r[h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W[h_t \odot h_{t-1}, x_t]), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

GRU vs LSTM in practice: GRU has fewer parameters (faster to train, less prone to overfitting on smaller datasets) and often achieves comparable performance to LSTM; LSTM’s separate cell state can give it a slight edge on tasks requiring very long-range memory. In practice, which one performs better is often dataset-dependent, and trying both is a reasonable default when it matters.

12.6 11.6 Word Embeddings

Before feeding text into any sequence model, words must become numeric vectors. **One-hot encoding** (a vector with a single 1 at the word’s index, 0 elsewhere) has two major problems: the vectors are enormous (one dimension per vocabulary word, often 50,000+) and *every* pair of words is equally “distant” from every other — there’s no notion that “king” and “queen” are more related than “king” and “banana.”

Word2Vec learns dense, lower-dimensional vectors (e.g., 300 dimensions) such that words appearing in similar contexts end up with similar vectors — trained via one of two setups: **CBOW** (predict a word from its surrounding context words) or **Skip-gram** (predict surrounding context words from a single word). The famous emergent property: vector arithmetic captures semantic relationships, e.g., $\text{vector}(\text{“king”}) - \text{vector}(\text{“man”}) + \text{vector}(\text{“woman”}) \approx \text{vector}(\text{“queen”})$ — this works because the training objective pushes semantically/contextually related words to have geometrically related vectors.

GloVe (Global Vectors): a related approach that learns embeddings by factoring a global word co-occurrence matrix (how often words appear near each other across the whole corpus), rather than Word2Vec’s local, sliding-window prediction task — both approaches tend to produce embeddings with broadly similar useful properties.

12.7 11.7 A First Look at Attention (Bridging Into Next Week)

Sequence-to-sequence RNN models (e.g., for machine translation) traditionally compress an entire input sequence into a single, fixed-size final hidden state (the “context vector”), which the decoder then uses to generate the entire output — this becomes an information bottleneck for longer sequences, since a fixed-size vector simply cannot losslessly represent an arbitrarily long sentence.

Attention’s fix: instead of relying solely on that single compressed vector, let the decoder, at *each* output step, look back at *all* of the encoder’s hidden states and learn to assign different weights (how much “attention” to pay) to each one, based on relevance to what it’s currently generating. This removes the fixed-size bottleneck and, as a side effect, tends to make long-range dependencies much easier to learn

than they are for a vanilla RNN — setting up exactly the idea that Transformers (next week) take to its logical extreme, discarding recurrence entirely in favor of attention.

12.8 Practice

1. By hand, trace one time step of a vanilla RNN's hidden state update, given small numeric W_{hh} , W_{xh} , h_{t-1} , and x_t .
2. Implement a vanilla RNN's forward pass in NumPy for a 5-step sequence, and empirically observe how the magnitude of $\frac{\partial h_5}{\partial h_1}$ shrinks compared to $\frac{\partial h_2}{\partial h_1}$ — connect this to the vanishing gradient explanation above.
3. Train an LSTM (using PyTorch's `nn.LSTM`) on a simple sequence classification task (e.g., sentiment classification on short movie reviews), and compare its validation accuracy to a vanilla `nn.RNN` on the same task and data.
4. Load pretrained GloVe or Word2Vec embeddings, and verify the “king - man + woman $\approx$ queen” relationship yourself using cosine similarity to find the nearest word vector to that computed result.

13 Week 12 — Transformers, NLP, and Project 2

13.1 12.1 Self-Attention — The Core Mechanism

The goal: for each position in a sequence, compute a new representation that’s a weighted combination of *every* position in the sequence (including itself), where the weights reflect how relevant each other position is to this one — letting the model directly relate any two positions, regardless of how far apart they are, in a single step (unlike an RNN, which must propagate information step by step through every position in between).

Query, Key, Value — where they come from and what they mean: each input embedding x_i is linearly projected (via three separate learned weight matrices) into three vectors:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i$$

Intuition: the **Query** represents “what this position is looking for”; the **Key** represents “what this position offers/advertises about itself”; the **Value** is “the actual content this position will contribute if attended to.” The Query-Key dot product measures how well what one position is looking for matches what another position offers.

The full computation:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Step by step: (1) compute QK^T — a matrix of dot products between every pair of positions’ queries and keys, measuring pairwise relevance; (2) divide by $\sqrt{d_k}$ (explained below); (3) apply softmax **row-wise**, turning each row into a probability distribution over all positions (how much to attend to each); (4) multiply by V — take a weighted sum of all positions’ value vectors, using the attention weights just computed.

Why divide by $\sqrt{d_k}$? For query/key vectors of dimension d_k with roughly independent, unit-variance entries, the dot product $q \cdot k$ has variance that grows proportionally with d_k — so as d_k increases, the raw dot products tend to have larger magnitude. Large-magnitude values pushed into softmax cause it to become extremely “peaked” (close to a one-hot vector), which in turn makes the softmax function’s gradient nearly zero almost everywhere — hurting learning. Dividing by $\sqrt{d_k}$ rescales the dot products back down to a range where softmax’s gradients remain informative and training proceeds smoothly.

Worked numeric intuition (simplified): imagine two positions where $q_1 \cdot k_1 = 2$ and $q_1 \cdot k_2 = 1$ (position 1 is more relevant to itself than position 2 is). Softmax of $[2, 1]$ gives roughly $[0.73, 0.27]$ — a fairly soft, informative distribution. If d_k were large and those same relative dot products scaled up to $[20, 10]$, softmax would output something like $[0.9999, 0.0001]$ — essentially one-hot, discarding nuance and killing gradient signal for the “losing” position.

13.2 12.2 Multi-Head Attention

Rather than computing a single attention operation, multi-head attention runs h independent attention computations in parallel, each with its *own* learned W_Q, W_K, W_V projections (typically each head operating in a smaller dimension, $d_k = d_{model}/h$, so the total compute stays comparable to a single full-size head):

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

Why multiple heads help: each head can specialize in learning a different *kind* of relationship between positions — empirically, different heads in a trained Transformer have been observed to focus on things like syntactic dependencies (e.g., a verb attending to its subject), coreference (a pronoun attending to the noun it refers to), or positional/local patterns — giving the model several independent “lenses” through which to relate positions, which are then combined.

13.3 12.3 Positional Encoding

Self-attention, as described above, computes a weighted sum over *all* positions with no inherent sense of order — if you shuffled the input sequence, the set of attention computations for each token relative to the others would be unchanged (attention operates on a set, not an ordered sequence, by default). Since word order obviously matters for language, **positional encodings** are added to each input embedding before it enters the attention layers, injecting information about each token’s position.

The original Transformer used fixed **sinusoidal** encodings — a combination of sine and cosine functions of different frequencies for each dimension — chosen partly because this makes it easy for attention to learn to attend by *relative* position (the encoding for position $p + k$ can be expressed as a linear function of the encoding for position p). Many modern models instead use **learned** positional embeddings (a trainable vector per position, just like a word embedding), which is simpler and works well in practice given enough training data.

13.4 12.4 The Full Transformer Block

Each Transformer layer combines multi-head self-attention with a position-wise feed-forward network (a simple 2-layer MLP applied independently to each position), wrapped with two key stabilizing techniques: - **Residual connections:** the input to a sub-layer (attention, or the feed-forward network) is added to that sub-layer’s output — exactly the same idea as ResNet’s skip connections (Week 10), for exactly the same reason: giving gradients a direct path backward through very deep stacks of layers. - **Layer normalization:** normalizes activations across the *feature* dimension (for each individual token/position, independent of the batch — unlike batch normalization, which normalizes across the batch dimension), stabilizing training in a way that’s particularly well-suited to variable-length sequence data.

13.5 12.5 Encoder-Only vs Decoder-Only Architectures

BERT (Bidirectional Encoder Representations from Transformers) — encoder-only: Uses full (bidirectional) self-attention — every position can at-

tend to every other position, both before and after it in the sequence. Pretrained via **Masked Language Modeling (MLM)**: randomly mask ~15% of input tokens and train the model to predict them using context from *both* directions — this bidirectional context is exactly what makes BERT strong at *understanding* tasks (sentence classification, named entity recognition, extractive question answering), where seeing the full context (including what comes after a given word) genuinely helps.

GPT (Generative Pre-trained Transformer) — decoder-only: Uses **causal (masked) self-attention** — each position can only attend to itself and earlier positions, never later ones (implemented by adding $-\infty$ to the attention scores for any “future” position before the softmax, forcing their weight to become 0). Pretrained via standard **autoregressive language modeling**: predict the next token given all previous tokens. This causal restriction is *necessary* for generation — at actual inference time, future tokens genuinely don’t exist yet, so the model must be trained under the same constraint it will face at inference, or there would be a mismatch between training and real use (“training/inference distribution shift”).

Why this architectural difference tracks the objective difference so directly: you cannot train a model to predict a word using bidirectional context and then expect it to generate text one token at a time using only leftward context at inference — the training signal wouldn’t match the deployment scenario. The masking pattern in self-attention is precisely what encodes which “cheating” (looking at information that won’t be available at inference/for the task) is disallowed.

13.6 12.6 Fine-Tuning with Hugging Face

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)

def tokenize(batch):
    return tokenizer(batch["text"], padding=True, truncation=True)

train_dataset = train_dataset.map(tokenize, batched=True)

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    learning_rate=2e-5,          # note: much smaller than typical from-scratch training
    evaluation_strategy="epoch",
)

trainer = Trainer(model=model, args=training_args, train_dataset=train_dataset, eval_dataset=eval_dataset)
trainer.train()
```

Why the learning rate is so small here: the model already contains extensive, useful pretrained knowledge — a large learning rate would apply large, disruptive updates that risk “catastrophically forgetting” that knowledge (overwriting well-learned general language understanding with noisy updates from your comparatively tiny fine-tuning dataset). A small learning rate makes gentle, targeted adjustments instead, adapting the pretrained representations to your specific task while mostly preserving what they already know.

Tokenization — why it’s not just “split on spaces”: modern Transformers use **subword tokenization** (e.g., WordPiece for BERT, Byte-Pair Encoding for GPT) — breaking rare/unknown words into smaller known subword pieces (e.g., “unbelievable” → “un”, “##believ”, “##able”). This keeps the vocabulary a manageable fixed size while still being able to represent essentially any word, including ones never seen during training, by composing it from familiar pieces.

13.7 12.7 Project 2 — NLP Project with a Pretrained Transformer

1. **Pick a concrete NLP task:** sentiment analysis, text classification, or summarization are all good, well-scoped choices.
2. **Load a pretrained model** appropriate to the task (an encoder model like BERT for classification, or a suitable pretrained summarization model) via Hugging Face.
3. **Fine-tune on your labeled dataset**, using a small learning rate as discussed above, and track training/validation loss to watch for overfitting (fine-tuning on a small dataset can overfit surprisingly fast).
4. **Evaluate with task-appropriate metrics:** accuracy/F1 for classification; ROUGE score for summarization.
5. **Deploy a minimal demo:** wrap the fine-tuned model in a small script or simple web app (e.g., using Gradio or Streamlit) so you can show, not just tell, what it does — this is a strong, concrete interview talking point.

13.8 Practice

1. By hand, on a tiny toy example (a 3-token sequence with small numeric embeddings), compute the QK^T matrix, apply the $\sqrt{d_k}$ scaling, run softmax row-wise, and compute the final attended output — this single worked-through example will cement the mechanism far better than reading about it alone.
2. Explain, without notes, why causal masking is necessary for GPT-style models but not for BERT-style models — tie the explanation directly to each model’s pretraining objective.
3. Fine-tune a pretrained BERT model (via Hugging Face) on a sentiment classification dataset, and report accuracy/F1 on a held-out test set.
4. Complete Project 2 as described in 12.7, and add it to your GitHub portfolio alongside Project 1.

14 Week 13 — DSA Foundations + Deployment Basics

14.1 13.1 Arrays and Strings

Arrays store elements contiguously in memory, giving $O(1)$ index access but $O(n)$ insertion/deletion in the middle (everything after must shift). Strings behave like arrays of characters, with the added wrinkle that in many languages (including Python) they're **immutable** — every “modification” actually creates a new string, which matters for the time complexity of code that repeatedly concatenates strings in a loop ($O(n^2)$ instead of the $O(n)$ you might assume).

Two pointers pattern: use two indices moving through the array (often from opposite ends, or both from the start at different speeds) to avoid nested loops.

```
def two_sum_sorted(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        s = arr[left] + arr[right]
        if s == target:
            return [left, right]
        elif s < target:
            left += 1          # need a bigger sum, move left pointer up
        else:
            right -= 1        # need a smaller sum, move right pointer down
    return None
```

This works *because the array is sorted* — moving left up can only increase the sum, moving right down can only decrease it, so you never need to backtrack. Time complexity: $O(n)$, versus $O(n^2)$ for checking every pair.

Sliding window pattern: maintain a window (subarray/substring) defined by two pointers, expanding and shrinking it as you scan through the array once, to avoid recomputing from scratch for every possible window.

```
def longest_unique_substring(s):
    seen = {}
    left = 0
    max_len = 0
    for right, char in enumerate(s):
        if char in seen and seen[char] >= left:
            left = seen[char] + 1          # shrink window past the repeat
        seen[char] = right
        max_len = max(max_len, right - left + 1)
    return max_len
```

14.2 13.2 Hashing

A hash map/set gives $O(1)$ *average-case* lookup, insert, and delete by computing a hash of the key to determine (roughly) where it's stored — this is the single most common tool for turning an $O(n^2)$ brute-force solution into $O(n)$.

```
def two_sum_unsorted(arr, target):
    seen = {} # value -> index
    for i, num in enumerate(arr):
        complement = target - num
        if complement in seen: # O(1) average lookup
            return [seen[complement], i]
        seen[num] = i
    return None
```

Compare this directly to the two-pointer version: this one works on an *unsorted* array in a single pass, trading $O(n)$ extra space for the hash map in exchange for not needing to sort first.

14.3 13.3 Sorting and Binary Search

Know the standard sorting algorithms' complexities well enough to justify a choice, without needing to reimplement them from scratch (Python's built-in `sorted()`/`.sort()` use Timsort, $O(n \log n)$): - **Merge sort:** $O(n \log n)$ guaranteed, stable, but uses $O(n)$ extra space. - **Quicksort:** $O(n \log n)$ average case, $O(n^2)$ worst case (rare with good pivot selection), in-place ($O(\log n)$ space for recursion).

Binary search isn't just "search a sorted array" — it applies to *any monotonic search space*, including searching for an answer *value* rather than an array index:

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Binary search on the answer (a pattern that surprises many beginners): for problems like "find the minimum capacity needed to ship packages within D days," you can binary search over the range of *possible answers* (capacities), checking at each midpoint whether that capacity is feasible (a monotonic yes/no check), rather than binary searching an array — recognizing this pattern turns many seemingly-unrelated "optimization" problems into a familiar binary search template.

14.4 13.4 Trees and Graphs — DFS and BFS

Depth-First Search (DFS) explores as far as possible down one path before backtracking — implemented via recursion (using the call stack implicitly) or an explicit stack.

```
def dfs(node, visited):
    if node is None or node in visited:
        return
    visited.add(node)
    process(node)
    for neighbor in node.neighbors:
        dfs(neighbor, visited)
```

DFS is the natural fit for: exploring all paths, detecting cycles, topological sorting, and backtracking-style problems (where you need to fully explore one branch before trying another).

Breadth-First Search (BFS) explores level by level, using a queue:

```
from collections import deque

def bfs(start):
    visited = {start}
    queue = deque([start])
    level = 0
    while queue:
        for _ in range(len(queue)):          # process one full level at a time
            node = queue.popleft()
            process(node, level)
            for neighbor in node.neighbors:
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)
        level += 1
```

BFS is the natural fit for: **shortest path in an unweighted graph** (the first time you reach a node via BFS is guaranteed to be via the shortest path, since you explore in strict order of distance), and level-order tree traversal.

Binary tree traversals — a foundational building block:

```
def inorder(node):          # left, root, right – gives sorted order for a BST
    if node:
        inorder(node.left)
        print(node.val)
        inorder(node.right)

def preorder(node):        # root, left, right – useful for copying/serializing a tree
    if node:
        print(node.val)
        preorder(node.left)
        preorder(node.right)
```

14.5 13.5 Dynamic Programming — How to Actually Recognize and Approach It

A problem is a DP candidate when it has: - **Overlapping subproblems**: the same smaller subproblem is solved repeatedly if you approach it with naive recursion (e.g., naive recursive Fibonacci recomputes `fib(2)` many times). - **Optimal substructure**: the optimal solution to the full problem can be built from optimal solutions to its subproblems.

The systematic approach (in order): 1. Define what a “state” represents in words (e.g., “the minimum cost to reach step i ”). 2. Write the **recurrence relation** — how does the state at i relate to smaller states? (e.g., `dp[i] = min(dp[i-1], dp[i-2]) + cost[i]`) 3. Identify the base case(s). 4. Decide: top-down (recursion + memoization — easier to write, closely mirrors the recursive definition) or bottom-up (iterative table filling — usually more space-efficient, avoids recursion depth limits).

Worked example — climbing stairs (you can take 1 or 2 steps at a time; how many distinct ways to reach step n ?): Recurrence: `ways(n) = ways(n-1) + ways(n-2)` — to reach step n , your last move was either a 1-step from $n - 1$, or a 2-step from $n - 2$; summing these covers every distinct path exactly once. Base cases: `ways(0)=1`, `ways(1)=1`. Notice this is structurally identical to the Fibonacci recurrence — recognizing that structural similarity across “different-looking” problems is a large part of what DP practice builds.

```
def climb_stairs(n):
    if n <= 1:
        return 1
    dp = [0] * (n + 1)
    dp[0], dp[1] = 1, 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

14.6 13.6 Model Serving Basics

A minimal FastAPI model server:

```
from fastapi import FastAPI
import joblib
import numpy as np

app = FastAPI()
model = joblib.load("model.pkl")    # load once at startup, not per-request

@app.post("/predict")
def predict(features: list[float]):
    x = np.array(features).reshape(1, -1)
    prediction = model.predict(x)
    return {"prediction": prediction.tolist()}
```

Why load the model once at startup, not inside the request handler: loading a model from disk can take significant time; doing it on every single request would add huge, unnecessary latency to every prediction — loading once and keeping it in memory for the life of the running server is the standard pattern.

A minimal Dockerfile for serving this app:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Why containerize at all: this Dockerfile captures the *exact* Python version and dependency versions the model was tested with — running the same image in production as in testing eliminates an entire category of bugs caused by environment mismatches (a different NumPy/scikit-learn version silently producing slightly different predictions is a real, hard-to-debug failure mode in ML specifically, more so than in typical software).

14.7 Practice

1. Solve, and state the time/space complexity of your solution: “given an array, find if any two numbers sum to a target” (both the $O(n^2)$ brute-force and the $O(n)$ hashing version).
2. Solve “find the length of the longest substring without repeating characters” using the sliding window pattern shown above; trace through a specific example by hand before coding it.
3. Implement DFS and BFS for a small graph represented as an adjacency list, and use BFS specifically to find the shortest path between two nodes.
4. Solve “0/1 knapsack” using the systematic DP approach in 13.5: define the state in words first, write the recurrence, identify base cases, then implement bottom-up.
5. Build the FastAPI server above around a real trained scikit-learn model, containerize it with the Dockerfile shown, and run it locally with `docker build + docker run`.

15 Week 14 — DSA Continued + Full ML Theory Review

15.1 14.1 More Tree and Graph Patterns

Binary Search Tree (BST) properties: for every node, all values in its left subtree are smaller, all values in its right subtree are larger. This means an **inorder traversal of a BST always produces values in sorted order** — a fact worth internalizing deeply, since it underlies many BST problems (e.g., “validate a BST” can be solved by checking that an inorder traversal is strictly increasing).

Common tree recursion pattern — “return info from children, combine at parent”:

```
def max_depth(node):
    if node is None:
        return 0
    return 1 + max(max_depth(node.left), max_depth(node.right))

def is_balanced(node):
    def check(node):
        if node is None:
            return 0
        left = check(node.left)
        if left == -1: return -1
        right = check(node.right)
        if right == -1: return -1
        if abs(left - right) > 1: return -1      # signal "unbalanced" upward
        return 1 + max(left, right)
    return check(node) != -1
```

This pattern — recurse on children, combine their results, return a value the parent will use — is the backbone of the large majority of tree problems.

Graph representations: adjacency list (a dict/array mapping each node to its neighbors — space-efficient for sparse graphs, the usual default) versus adjacency matrix (an $n \times n$ grid of connections — $O(1)$ edge lookup but $O(n^2)$ space, reasonable only for dense graphs or when you specifically need fast edge-existence checks).

Topological sort: an ordering of nodes in a Directed Acyclic Graph (DAG) such that every edge points from an earlier node to a later one in the ordering — used for problems like “course scheduling given prerequisites.” Typically implemented via DFS (append nodes to the result as you finish exploring them, then reverse) or Kahn’s algorithm (repeatedly remove nodes with no remaining incoming edges, using a queue).

15.2 14.2 Backtracking

The pattern: build a solution incrementally, and whenever you determine the current partial solution *cannot* lead to a valid complete solution, abandon it and back-track (undo the last choice) rather than continuing — this pruning is what makes

backtracking far more efficient in practice than exhaustively generating every possibility and checking validity at the end.

```
def subsets(nums):
    result = []
    def backtrack(start, path):
        result.append(path[:])          # record every partial state as a valid sub
        for i in range(start, len(nums)):
            path.append(nums[i])         # choose
            backtrack(i + 1, path)       # explore
            path.pop()                  # un-choose (backtrack)
        backtrack(0, [])
    return result
```

The three-part rhythm to recognize in every backtracking solution: choose (add something to the current partial solution), explore (recurse deeper), un-choose (remove it before trying the next option) — this exact choose/explore/un-choose pattern appears in permutations, combinations, N-Queens, and Sudoku solvers alike, just with different validity/pruning conditions.

15.3 14.3 Complexity Analysis — A Practical Checklist

When you finish any solution, be ready to state: - **Time complexity:** count the dominant operation. A single loop over n elements $\rightarrow O(n)$. Nested loops over the same input $\rightarrow O(n^2)$. Halving the search space each step (binary search) $\rightarrow O(\log n)$. Sorting first, then a linear pass $\rightarrow O(n \log n)$. - **Space complexity:** don't forget recursion's implicit stack space — a recursive solution with depth n uses $O(n)$ space even if it allocates no explicit data structures. - **Best/average/worst case, when they differ meaningfully:** e.g., hashing is $O(1)$ *average* but $O(n)$ *worst case* (in the rare event of many collisions) — worth mentioning if directly relevant to the problem's constraints.

15.4 14.4 ML Theory — Full Review, Interview-Ready

This section consolidates the “why” behind everything you’ve learned since Week 4, phrased as the direct explanations you should be able to give from memory.

Bias-variance tradeoff: total expected prediction error decomposes as $\text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$. Bias is error from a model too simple to capture the true underlying pattern (underfitting). Variance is error from a model too sensitive to the specific training sample it happened to see (overfitting — it's capturing noise, not signal). Simple models (linear regression, shallow trees) tend toward high bias/low variance; complex models (deep trees, large neural nets, high-degree polynomials) tend toward low bias/high variance. There's no way to minimize both simultaneously by adjusting complexity alone — you're choosing a point on that tradeoff curve, and the best point depends on how much data you have (more data generally lets you afford lower bias/more complexity without paying as much variance cost).

Why gradient descent, not just “solve for the minimum directly”: for most ML models (anything beyond simple linear regression), there's no closed-form solution

for the loss-minimizing parameters — the loss surface is too complex. Gradient descent instead iteratively nudges parameters in the direction that locally decreases loss fastest, repeating until convergence — a general-purpose optimization method that works for essentially any differentiable loss function and model.

Backpropagation, one more time, compressed: the chain rule applied systematically, working backward from the output loss through every layer, reusing each layer’s computed gradient to compute the next one backward — this reuse is exactly what makes it computationally tractable for networks with millions of parameters.

Why regularization works: by penalizing large weights, you constrain the model’s effective complexity/flexibility, pushing it away from perfectly memorizing training-set noise and toward simpler solutions that are more likely to generalize — this is a direct, deliberate move toward the “less variance” end of the bias-variance tradeoff, at the cost of slightly more bias.

Precision/recall tradeoff, stated generally: every classifier with a continuous output (a probability) faces a tradeoff as you move its decision threshold — lowering the threshold catches more true positives (higher recall) but also more false positives (lower precision); raising it does the reverse. There is no threshold that’s universally “correct” — the right choice depends entirely on the relative real-world cost of false positives versus false negatives for your specific problem.

Ensembles, one more time, compressed: bagging trains independent models on resampled data and averages, reducing variance by cancelling out weakly-correlated errors, while leaving each model’s inherent bias unchanged. Boosting trains models sequentially, each correcting the ensemble’s current errors, directly reducing bias (at some risk of increasing variance if run too long without regularization).

Why CNNs for images, RNNs/Transformers for sequences — the underlying principle in both cases: good architectures build in the right **inductive bias** — structural assumptions matched to the data’s actual structure. CNNs assume local spatial patterns matter and can appear anywhere (parameter sharing across space). RNNs assume sequential/temporal order matters and process it step by step. Transformers assume relationships between *any* pair of positions can matter, regardless of distance, and compute all pairwise relationships directly via attention. Choosing an architecture is choosing which inductive bias matches your data.

Why Transformers largely displaced RNNs for NLP: (1) self-attention computes relationships between all pairs of positions in parallel, not sequentially — much faster to train on modern parallel hardware; (2) direct pairwise connections regardless of distance sidestep the vanishing-gradient-driven long-range dependency struggles that RNNs face.

15.5 Practice

1. Solve a topological sort problem (e.g., “course schedule” — can all courses be finished given prerequisites) using DFS-based ordering.
2. Solve the classic “N-Queens” problem using the backtracking choose/explore/unchoose pattern described above.

3. Without any notes, write out (in your own words, on paper) the bias-variance tradeoff, why cross-entropy is used over MSE for classification, why ResNet's skip connections help, and why attention removed the RNN bottleneck — then check what you wrote against the explanations in this book and previous weeks.
4. Build your own one-page “cheat sheet” — not by copying this book's, but by writing your own condensed version from memory, then comparing against the book's cheat sheet (Part VIII equivalent) to see what you missed.

16 Week 15 — ML System Design + Portfolio

16.1 15.1 Why System Design Is a Distinct Skill From Modeling

Everything through Week 14 trains you to build a *model*. System design trains you to build a *system* — the surrounding infrastructure of data pipelines, serving, monitoring, and iteration that turns a model into something that reliably works in the real world, at scale, over time. This is frequently the actual day-to-day work of an ML engineer, more so than novel model architecture design.

16.2 15.2 A Structured Framework You Can Reuse for Any Design Problem

Step 1 — Clarify the actual requirements. Never start designing before you know: What’s the business objective? What does “success” mean, measured how? What’s the scale (queries per second, data volume)? What’s the latency budget (real-time, under 100ms? or batch, overnight is fine)? What constraints exist (compute budget, privacy/compliance requirements)? Asking these out loud, even in an interview, demonstrates senior-level thinking — jumping straight to “I’ll use a neural network” without this step is a common, avoidable mistake.

Step 2 — Translate the business goal into a concrete ML problem formulation. “Reduce customer churn” isn’t an ML task by itself — it becomes one once you frame it as, e.g., “predict the probability a customer cancels within the next 30 days, and intervene on the highest-risk segment.” This translation step is where a lot of real-world ML value (and a lot of interview signal) actually lives.

Step 3 — Data. What data exists, and what would need to be newly collected or labeled? How fresh does it need to be? What biases might exist in how the data was collected (e.g., you only have data on customers who didn’t already churn before you started tracking them)?

Step 4 — Feature engineering, and the batch/real-time distinction. Some features can be precomputed in a batch pipeline (e.g., “average order value over the last 90 days,” updated nightly); others need to be computed in real time from live signals (e.g., “items currently in cart”). This distinction has major implications for system architecture and for the train-serve skew risk discussed in Week 13’s spirit — the same computation logic must be used in both training and serving, or the model will behave unpredictably in production despite looking fine offline.

Step 5 — Model selection, with explicit tradeoffs. Always justify complexity, don’t default to it: “I’d start with logistic regression or gradient boosting as a baseline — it’s fast to train, cheap to serve, and interpretable, which matters here because [reason]. I’d only move to a deep learning approach if [specific reason: e.g., raw unstructured input like images/text, or the baseline’s accuracy ceiling is a genuine business blocker].” This kind of reasoning, not just knowing that deep learning exists, is what a system design interview is testing.

Step 6 — Evaluation, offline and online. Offline metrics (AUC, precision/recall, RMSE) tell you how the model performs on historical data — necessary but not suffi-

cient. **Online evaluation (A/B testing)** tells you how it actually affects the business metric you care about, because offline metrics can improve while the real-world outcome doesn't (e.g., a recommendation model with better offline accuracy that somehow reduces actual user engagement due to reduced diversity of recommendations).

Step 7 — Deployment strategy. Rarely should a new model replace the old one instantly for 100% of traffic. **Canary deployment** (route a small percentage of traffic to the new model, watch closely, gradually increase) or a formal **A/B test** (split traffic, compare metrics with statistical rigor) reduce the blast radius of an unexpected regression.

Step 8 — Monitoring and the iteration loop. Track input feature distributions (data drift), prediction distributions, and — once available — actual outcome-based performance metrics. Define a retraining cadence and a feedback loop: how do today's predictions and their eventual outcomes become tomorrow's training data?

16.3 15.3 Worked Case Study — Recommendation System

Requirements clarification: platform type (video/e-commerce/social), scale (millions of items, millions of users), latency budget (typically must respond in well under 100ms for a homepage feed).

Problem formulation: predict a relevance/engagement score for each (user, item) pair; rank items by that score.

The fundamental scale problem, and its standard solution — a two-stage funnel: Scoring every one of potentially millions of items for every user request with a complex, accurate model is computationally infeasible at real-time latency. The standard architecture splits this into two stages: 1. **Candidate generation (retrieval):** a fast, cheaper method narrows millions of items down to a few hundred plausible candidates — often via approximate nearest-neighbor search over learned embeddings (represent users and items as vectors such that relevant pairs are close together in embedding space; use an efficient approximate search index like FAISS or HNSW to find the nearest item-vectors to a user-vector quickly, without scanning every item). 2. **Ranking:** a more powerful, more expensive model (which can afford to use much richer features, since it only needs to score a few hundred candidates, not millions) scores and orders just that shortlist.

Feature considerations: user features (long-term preferences, recent activity), item features (category, popularity, freshness), and crucially, **context features** (time of day, device) and cross features (user-item interaction history).

The feedback loop risk, and why it matters here specifically: if the model only ever learns from what users clicked, and users can only click what the model already chose to show them, the system can enter a self-reinforcing loop that never explores potentially-relevant items it initially ranked low — some degree of deliberate exploration/randomization in what's shown is often needed to keep the training data itself unbiased over time.

16.4 15.4 Worked Case Study — Fraud Detection

Key characteristics that shape the entire design: extreme class imbalance (fraud might be 0.1% of transactions); very tight latency requirement (a decision is often needed within milliseconds, at the moment of transaction); highly asymmetric costs (a missed fraud case can be very expensive; but blocking a legitimate customer also has a real cost — lost trust, lost revenue); and an **adversarial** setting — unlike most ML problems, the “negative class” here is actively trying to evade detection and adapts its behavior over time in response to the model’s decisions.

Design implications that follow directly from these characteristics: given the class imbalance, accuracy is useless as a metric (Week 3’s Bayes’ theorem worked example is the exact underlying reason); precision/recall, and the specific tradeoff point, must be chosen based on real dollar-cost estimates of false positives versus false negatives. Given the adversarial nature, the model needs a fast retraining/monitoring cadence — a model that was excellent last month can degrade quickly as fraud patterns shift, so this system needs tighter drift monitoring than a more static prediction task.

16.5 15.5 What Interviewers Are Actually Listening For

Not one specific “correct” architecture — there generally isn’t one. They’re listening for: whether you clarify ambiguity before diving in; whether you can explicitly state tradeoffs rather than presenting a single design as obviously correct; whether your technical choices trace back to the actual business objective you established in Step 1; and whether you treat deployment as the beginning of ongoing work (monitoring, iteration), not the finish line.

16.6 15.6 Portfolio and Resume

GitHub: pin your best 2–3 projects. Each needs a README that a stranger can understand in under two minutes: problem, approach, results, and (importantly) what you’d improve with more time — this last part signals genuine understanding, not just following a tutorial.

Resume: quantify impact where you can, even on personal projects (“improved F1 from 0.71 to 0.84 by adding XGBoost and engineered features” beats “built a machine learning model”). Mirror the specific keywords/skills in the job descriptions you’re targeting, without fabricating experience.

16.7 Practice

1. Pick one case study above (recommendation system or fraud detection) and write out a full answer using the 8-step framework, entirely from memory, before rereading this chapter.
2. Design a third system from scratch using the framework: a content moderation / spam detection system, explicitly addressing its adversarial nature and its precision/recall tradeoff.

3. Rewrite your two Project READMEs (from Weeks 8 and 12) against the “stranger, two minutes” standard above — have someone unfamiliar with the project read it and tell you what’s unclear.
4. Tailor your resume’s bullet points to match the language of three real job postings for roles you’re targeting.

17 Week 16 — Mock Interviews, Final Review, and Applications

17.1 16.1 Why This Week Is Different

There's no new technical content this week — by now you've covered everything from Python through system design. The job now is **retrieval practice**: proving to yourself (and soon, to interviewers) that you can produce this knowledge under time pressure, out loud, without notes. Recognition (reading an explanation and nodding along) and recall (producing the explanation yourself from a blank page) are very different skills, and only the second one is what an interview actually tests.

17.2 16.2 Structuring a Mock Interview

Run each mock as three back-to-back segments, the way a real interview loop typically is: 1. **A short coding problem** (20–30 min) — think out loud, state a brute-force approach first, then optimize, then test against edge cases. Use the patterns from Weeks 13–14: identify which pattern fits before writing code. 2. **ML theory / rapid-fire questions** (15–20 min) — have a partner (or yourself, from a written list) fire off questions like the ones throughout this book's weekly chapters, and answer out loud without notes. 3. **A system design question** (20–30 min) — pick one you haven't specifically pre-memorized an answer for, and work through the Week 15 framework live.

If you don't have a study partner, use a platform like Pramp or interviewing.io for peer mock interviews, or record yourself answering questions out loud and review the recording critically — you'll often notice hesitations or gaps you didn't feel while speaking.

17.3 16.3 Behavioral Interview Prep — The STAR Method

Most loops include at least one behavioral round, and “tell me about a project” or “tell me about a challenge you faced” questions are near-universal. Structure every answer as: - **Situation**: brief context — what was the project/problem? - **Task**: what specifically were you responsible for? - **Action**: what did *you* actually do — be specific about your reasoning and decisions, not just what happened. - **Result**: what was the outcome, ideally with a concrete number or clear before/after?

Prepare 4–5 stories in advance (from your two portfolio projects, and any other relevant experience) that can each flex to answer several different behavioral questions (“a time you solved a hard technical problem,” “a time you had to make a tradeoff under a deadline,” “a time your first approach failed and you had to change direction”) — you don't need a unique story for every possible question, you need a handful of strong, detailed ones you can angle differently.

17.4 16.4 A Consolidated Rapid-Review Pass

Before each mock or real interview, do a fast pass through these, entirely from memory:

Foundations: bias-variance tradeoff; overfitting fixes; train/val/test split and why the test set is touched once; supervised vs unsupervised vs reinforcement learning.

Classical ML: why cross-entropy over MSE for classification; L1 vs L2 regularization and why L1 zeros out weights; bagging vs boosting (what each reduces, and why); precision vs recall and when each matters more; why KNN/SVM need feature scaling but trees don't.

Deep learning: why non-linear activations are required; why ReLU is preferred over sigmoid in hidden layers; vanishing/exploding gradients and their fixes; why ResNet's skip connections help; why LSTM handles long sequences better than vanilla RNN; the scaled dot-product attention formula and why the $\sqrt{d_k}$ scaling exists; BERT vs GPT and how their masking pattern follows from their training objective.

Math: what eigenvectors/eigenvalues mean and how they relate to PCA; the chain rule and its role in backpropagation; Bayes' theorem and the classic "rare disease" worked example, and how it explains why accuracy is misleading on imbalanced data; what a p-value actually means (and the common misinterpretation to avoid).

Systems and deployment: train-serve skew and why it happens; data drift vs concept drift; why canary/A-B rollouts are used instead of instant full deployment; the two-stage (retrieval + ranking) pattern for recommendation systems at scale.

DSA patterns: two pointers, sliding window, hashing, binary search (including "binary search on the answer"), DFS/BFS, the choose/explore/un-choose backtracking rhythm, and how to recognize a DP problem (overlapping subproblems + optimal substructure) and derive its recurrence.

If any of these produce hesitation rather than a fluent answer, that's your signal for where to spend your remaining review time — go back to that specific week's chapter, not a general re-read of everything.

17.5 16.5 Applying — Don't Wait for "Fully Ready"

Interview pipelines take real calendar time to move through (screening calls, technical rounds, offer negotiation can easily span 3-6+ weeks per company). If you only start applying once you feel "100% ready," you lose that lead time and end up job-searching for months *after* your study plan ends, rather than landing an offer close to when your prep concludes. Applying, and interviewing, is itself part of the learning process — you will get real questions this book didn't specifically cover, and each interview (successful or not) sharpens the next one.

A reasonable posture for this final week: keep applying broadly, treat early interviews partly as calibration (what's actually being asked, where are your specific gaps), and fold what you learn from them directly back into your rapid-review pass above.

17.6 Practice

1. Do a full mock interview (coding + theory + system design) with a peer or on a mock-interview platform, and write down every question you hesitated on after-

ward.

2. Write out your 4-5 STAR-method stories in full, then have someone else read them and ask you a follow-up question on each — practicing extending your answer under a follow-up is as important as the initial answer.
3. Go through the rapid-review list in 16.4 out loud, timing yourself — flag anything that took more than a few seconds to start answering fluently, and revisit that week's chapter today.
4. Submit at least 5 real applications this week, tailoring your resume bullets (from Week 15) to each posting's specific language.

18 Closing

You’ve now covered, in real depth, everything from Python fundamentals through Transformer architecture, classical ML through system design, and the coding patterns that show up in technical screens. That’s the complete, honest shape of what a 4-month AI/ML Engineer path actually requires — not a shortcut version of it.

The material in this book will still be here after you land the role. Come back to specific chapters whenever something at work doesn’t quite make sense yet — that’s exactly what it’s for.